



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 804
Группа М8О-404Б-22 Направление подготовки 01.03.02 «Прикладная математика и
информатика»

Профиль Прикладная математика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему: Разработка рекомендательной системы на основе отзывов
пользователей маркетплейсов

Автор ВКРБ:	Чёрный Дмитрий Александрович	(_____)
Руководитель:	Лебедев Максим Витальевич	(_____)
Консультант:	—	(_____)
Консультант:	—	(_____)
Рецензент:	—	(_____)

К защите допустить

Заведующий кафедрой № 806	Крылов Сергей Сергеевич	(_____)
____ мая 2026 года		

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 90 страниц, 9 рисунков, 1 таблицы, 14 использованных источников, 1 приложения.

РЕКОМЕНДАТЕЛЬНАЯ СИСТЕМА, РЕКОМЕНДАЦИИ, АНАЛИЗ ОТЗЫВОВ, ОБРАБОТКА ЕСТЕСТВЕННОГО ЯЗЫКА, ПЕРСОНАЛИЗАЦИЯ, НЕЙРОСЕТИ, PYTHON

В данной работе реализуется рекомендательная система семантического поиска товаров, которая по произвольному текстовому запросу пользователя возвращает ранжированный список релевантных товаров без необходимости указания точных ключевых слов или категории.

В теоретической части работы рассматриваются основные подходы к построению рекомендательных систем в электронной коммерции: коллаборативная фильтрация, контентная фильтрация и гибридные системы. Приводится анализ роли пользовательских отзывов как источника семантической информации о товарах и их преимуществ перед явными числовыми оценками. Описываются меры близости текстовых векторов, применяемые в задачах информационного поиска, а также метрики оценки качества классификации.

В практической части описывается построение рекомендательной системы на основе открытых датасетов маркетплейса Wildberries. Проводится сравнительный анализ языковых моделей — Word2Vec, text2vec и трансформерных моделей RuBERT, MiniLM и multilingual-E5-large на базе библиотеки Sentence Transformers — по критерию качества классификации отзывов по товарным категориям. Описывается реализация трёх методов классификации и семантического поиска: логистической регрессии, центроидного классификатора и сиамской нейронной сети с метрическим обучением. Проводится оценка качества реализованных методов с использованием взвешенной F1-меры в зависимости от объёма обучающей выборки и числа товарных категорий.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	5
ВВЕДЕНИЕ	6
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	9
1.1 Определение рекомендательной системы	9
1.2 Цели и задачи рекомендательных систем	9
1.2.1 Снижение информационной перегрузки покупателя	10
1.2.2 Увеличение конверсии и среднего чека	10
1.2.3 Удержание пользователя за счёт релевантных рекомендаций	11
1.3 Отличие рекомендательных систем от поисковых	11
1.4 Классификация рекомендательных систем	12
1.4.1 Коллаборативная фильтрация (Collaborative Filtering)	13
1.4.2 Контентная фильтрация (Content-based Filtering)	14
1.4.3 Гибридные системы	15
1.5 Отзывы как источник информации о предпочтениях	16
1.5.1 Отличие от явных оценок	16
1.5.2 Влияние пользовательских отзывов на работу алгоритмов рекомендаций	17
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	19
2.1 Подготовка и выбор технологий	20
2.1.1 Выбор языка программирования Python для реализации системы	20
2.1.2 Jupyter Notebook как среда итеративной разработки	21
2.1.3 Поиск данных и структура выбранных датасетов	21
2.1.4 Загрузка данных: установка и настройка Docker, развёртывание PostgreSQL и заполнение базы данных	25
2.2 Реализация загрузки данных	29
2.2.1 Таблицы отзывов, товаров и обработка датасетов	29
2.2.2 Качественное описание датасетов	32
2.3 Реализация рекомендательной системы	35
2.3.1 Меры близости текстовых векторов	35
2.3.2 Задача классификации как критерий пригодности модели эмбедингов. Модели Word2Vec, Text2Vec	38
2.4 Методы классификации отзывов и поиска товаров	42

2.4.1	Метод А — логистическая регрессия	42
2.4.2	Метод Б — центроидный классификатор	44
2.4.3	Метод В — сиамская нейронная сеть	46
2.5	Разработка графического интерфейса	48
2.6	Результаты решения задачи классификации отзывов и поиска релевантных товаров	52
2.6.1	Результаты поиска релевантных товаров	52
2.6.2	Результаты классификации отзывов	56
2.6.3	Перспективы развития	67
ЗАКЛЮЧЕНИЕ		70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		72
ПРИЛОЖЕНИЕ А Приложение с кодом		74

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

API — application programming interface

CI/CD — continuous integration/continuous deployment

STOMP — streaming text oriented messaging protocol

ЯП — язык программирования

ОС — операционная система

СУБД — система управления базами данных

БД — база данных

SQL — Structured Query Language

JSON — JavaScript Object Notation

ВВЕДЕНИЕ

Стремительное развитие электронной коммерции и маркетплейсов привело к тому, что современные торговые платформы предлагают пользователям миллионы товарных позиций одновременно. В условиях столь широкого ассортимента традиционные методы поиска, основанные на совпадении ключевых слов, перестают справляться с задачей персонализированного подбора товаров: пользователь, введя запрос «белые кроссовки», получает тысячи результатов, среди которых ему предстоит самостоятельно ориентироваться. Эта проблема информационной перегрузки является одним из ключевых вызовов современной электронной торговли и служит главной мотивацией для разработки интеллектуальных рекомендательных систем.

Рекомендательные системы представляют собой программные инструменты, анализирующие предпочтения и поведение пользователей с целью формирования персонализированных предложений товаров, услуг или контента. В отличие от поисковых систем, которые отвечают на явно сформулированный запрос и возвращают одинаковые результаты для всех пользователей, рекомендательные системы предсказывают скрытые интересы — предлагают пользователю то, что он ещё не искал, но с высокой вероятностью хотел бы найти. Именно эта способность к персонализации без явного запроса делает рекомендательные системы незаменимым инструментом в арсенале крупных торговых платформ. Классические подходы к построению рекомендательных систем — коллаборативная фильтрация, основанная на схожести поведения пользователей, и контентная фильтрация, основанная на сходстве характеристик товаров, — хорошо изучены и широко применяются. Однако оба подхода в чистом виде обладают существенными ограничениями: коллаборативная фильтрация страдает от проблемы холодного старта для новых пользователей и товаров, а контентная — от склонности к сверхспециализации и зависимости от полноты структурированных метаданных.

Особую роль в преодолении этих ограничений играют пользовательские отзывы. Текст отзыва содержит развёрнутую семантическую информацию о товаре, недоступную в структурированных атрибутах каталога: реальные впечатления от использования, упоминание конкретных свойств, сравнения с

аналогами, эмоциональную окраску. Фраза «подошва мягкая, но быстро стирается» несёт принципиально больше информации для рекомендательной системы, чем числовая оценка «три звезды». Совокупность тысяч таких отзывов на один товар формирует его семантический портрет, который может быть использован как для классификации по категориям, так и для поиска по смысловому сходству с запросом пользователя. На крупных маркетплейсах, таких как Wildberries, объём накопленных отзывов исчисляется сотнями миллионов — это колоссальный информационный ресурс, который традиционные системы поиска по ключевым словам практически не используют.

Настоящая работа посвящена разработке рекомендательной системы, использующей тексты пользовательских отзывов в качестве основного источника информации для решения двух взаимосвязанных задач: автоматической классификации отзыва по товарной категории и семантического поиска товаров по произвольному текстовому запросу.

Задача классификации отзывов по товарным категориям служит критерием пригодности языковой модели — модель, формирующая информативные векторные представления текстов, обеспечивает высокое качество как в классификации, так и в поиске. Конечная цель системы — по тексту, введённому пользователем, вернуть ранжированный список товаров, семантически близких к запросу, без необходимости указания ключевых слов.

Данные для обучения и тестирования системы получены из открытых источников — публично доступных датасетов маркетплейса Wildberries, содержащих тексты отзывов покупателей, числовые оценки товаров и структурированные описания товарных позиций. Хранение и обработка данных организованы на основе реляционной базы данных PostgreSQL, развёрнутой в среде контейнеризации Docker. Использование Docker обеспечивает полную воспроизводимость окружения: база данных, её настройки и схема таблиц фиксируются в конфигурационном файле, что позволяет развернуть идентичную инфраструктуру на любой машине без ручной настройки. База данных содержит две ключевые таблицы: таблицу товаров с полями артикула, названия, корневой категории и текстового описания, и таблицу отзывов с текстом отзыва, числовой оценкой и привязкой к артикулу товара.

Разработка рекомендательных систем на основе текстовых данных

сопряжена с рядом практических трудностей, которые необходимо учитывать при проектировании подобных систем. Одной из ключевых проблем является несбалансированность классов в обучающих данных. На реальных маркетплейсах одни товарные категории представлены тысячами отзывов, тогда как другие — лишь единицами. При обучении классификатора на несбалансированных данных модель склонна игнорировать редкие классы, максимизируя общую точность за счёт частых категорий. Это приводит к тому, что система корректно распознаёт популярные категории, но практически не справляется с малочисленными. Существенным вызовом является также высокая вычислительная стоимость векторизации текстовых данных. Современные трансформерные языковые модели обеспечивают высокое качество семантических представлений, однако обработка десятков тысяч текстов занимает значительное машинное время. Отдельную сложность представляет работа с большими объёмами данных в условиях ограниченных ресурсов оперативной памяти. Загрузка сотен тысяч записей из базы данных единым запросом может привести к исчерпанию доступной памяти и аварийному завершению процесса. Для решения каждой из перечисленных проблем в работе предложены и реализованы конкретные технические решения, описанные в практической части.

Подобная рекомендательная система находит применение в широком круге задач электронной коммерции. На крупных маркетплейсах она позволяет формировать персонализированные подборки товаров на основе текстовых описаний потребностей пользователя без необходимости знать точные названия или артикулы. В системах управления ассортиментом розничных сетей семантический поиск упрощает навигацию по каталогу с миллионами позиций. В аналитических системах мониторинга качества товаров автоматическая классификация отзывов позволяет оперативно выявлять проблемные категории. В системах сравнения товаров предложенный подход позволяет сопоставить описание искомого товара с имеющимися в каталоге позициями без точного знания ключевых слов. Накопленная база векторных представлений отзывов открывает перспективы для построения полноценной рекомендательной системы, учитывающей аспектные предпочтения пользователей: не только категории товаров, которые они покупают, но и конкретные свойства, которые они ценят.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Определение рекомендательной системы

Рекомендательная система представляет собой программный инструмент, предназначенный для анализа предпочтений и поведения пользователей с целью формирования персонализированных предложений товаров, услуг или контента. Согласно классическому определению, рекомендательные системы — это программные средства и технологии, обеспечивающие пользователю подсказки относительно объектов, которые могут оказаться для него полезными [1]. Понятие «объект» здесь используется в широком смысле: это может быть книга, музыкальный трек, новостная статья, туристический маршрут, вакансия или товар в интернет-магазине. В более широкой трактовке, рекомендательной системой можно считать любую систему, которая формирует индивидуализированные рекомендации или направляет пользователя персонализированным образом. Ключевым признаком является именно персонализация: разные пользователи получают разные рекомендации даже при одинаковых условиях запроса. Рекомендательная система состоит из трёх взаимосвязанных модулей. Первый — модуль моделирования пользователя, который накапливает, хранит и обновляет информацию о его интересах и предпочтениях. Второй — модуль моделирования объектов рекомендации, описывающий характеристики каждого предлагаемого элемента. Третий — модуль реализации алгоритма рекомендаций, который на основе сопоставления пользовательской модели с характеристиками объектов вычисляет и формирует итоговый список предложений. Совокупная работа этих трёх модулей и составляет суть рекомендательного процесса.

1.2 Цели и задачи рекомендательных систем

Рекомендательные системы одновременно решают задачи двух сторон: торговой платформы и конечного пользователя. Несмотря на разную специфику, правильно выстроенная система заставляет их работать на общий результат.

1.2.1 Снижение информационной перегрузки покупателя

Основная проблема, которую решают рекомендательные системы, — информационная перегрузка. Современные маркетплейсы предлагают миллионы товарных позиций. Крупный интернет-магазин, подобный Amazon, содержит сотни миллионов наименований, что делает физически невозможным самостоятельный просмотр даже малой их части. Пользователь, столкнувшийся с избыточным выбором, испытывает когнитивную перегрузку: вместо того чтобы приносить удовлетворение, широкий ассортимент начинает восприниматься как источник стресса и неопределённости.

Рекомендательная система решает эту проблему, выступая в роли персонального фильтра: из всего множества доступных объектов она отбирает и предъявляет пользователю лишь те, которые с наибольшей вероятностью соответствуют его текущим потребностям. Тем самым система не ограничивает свободу выбора, а делает этот выбор управляемым и осмысленным.

Важно подчеркнуть, что рекомендации возникают не только тогда, когда пользователь явно что-то ищет. Значительная часть рекомендательных сценариев носит проактивный характер: система предлагает релевантные объекты без явного запроса со стороны пользователя, ориентируясь на его поведенческую историю, контекст текущей сессии или демографический профиль.

1.2.2 Увеличение конверсии и среднего чека

С точки зрения торговой платформы, рекомендательная система выполняет несколько коммерческих функций. Прежде всего, она увеличивает конверсию — долю посетителей, совершивших покупку. Показывая пользователю товары, соответствующие его реальным интересам, система повышает вероятность того, что он найдёт что-то нужное и оформит заказ.

Помимо этого, рекомендательные системы расширяют продажи за счёт так называемого «длинного хвоста» ассортимента. Без рекомендательной системы платформе экономически невыгодно продвигать малопопулярные товары: бюджет рекламы направляется на хиты продаж. Рекомендательная система решает эту проблему иначе: она рекомендует нишевые товары именно тем пользователям, которые с высокой вероятностью заинтересованы

в них [2], не тратя ресурсы на широкую аудиторию. В результате продажи равномернее распределяются по всему каталогу.

Наконец, рекомендации на страницах товара («с этим товаром также покупают») и в корзине («вам может понравиться») увеличивают средний чек за счёт кросс-продаж и допродаж сопутствующих позиций.

1.2.3 Удержание пользователя за счёт релевантных рекомендаций

Долгосрочная ценность рекомендательных систем для платформы определяется не только мгновенными продажами, но и формированием лояльности пользователей. Система, которая с каждым визитом «узнаёт» пользователя лучше и предлагает ему всё более точные рекомендации, создаёт устойчивое преимущество: пользователю незачем искать альтернативы, поскольку привычный сервис понимает его потребности.

Этот механизм работает через накопление пользовательской модели: чем дольше пользователь взаимодействует с системой, тем полнее его профиль предпочтений и тем точнее рекомендации. Так пользователю становится сложнее переключиться на новую платформу, где придётся начинать накопление истории взаимодействий заново.

1.3 Отличие рекомендательных систем от поисковых

Рекомендательные и поисковые системы нередко воспринимаются как близкие технологии, поскольку обе помогают пользователю находить релевантную информацию среди большого объёма данных. Однако между ними существует принципиальное различие, определяющее сферу применения и логику работы каждой из них.

Поисковая система отвечает на явный запрос: пользователь формулирует потребность в виде ключевых слов, и система возвращает документы, наиболее соответствующие этому запросу. При этом результаты для одного и того же запроса в классической поисковой системе будут одинаковыми для всех пользователей — система оптимизирует релевантность текста, а не соответствие конкретному человеку.

Рекомендательная система, напротив, работает со скрытыми интересами: она предсказывает, что может понравиться пользователю, даже если он сам этого ещё не знает и явно не запрашивал. Пользователь не

формулирует запрос — система самостоятельно анализирует его поведение, историю покупок, оценки и строит предположение о его предпочтениях.

Это различие можно проиллюстрировать следующим образом. Пользователь, введя в поисковой строке «кроссовки Nike белые 42 размер», получит набор результатов, одинаковых для всех, кто ввёл тот же запрос. Рекомендательная система, не дожидаясь запроса, предложит этому пользователю кроссовки на основе того, что он просматривал ранее, какие товары покупал, какие категории его интересуют — то есть на основе модели его поведения.

Следует, однако, отметить, что граница между поиском и рекомендацией постепенно размывается. Современные поисковые системы внедряют элементы персонализации: один и тот же запрос может давать разные результаты для разных пользователей с учётом их истории поиска и местоположения. В этом смысле персонализированный поиск может рассматриваться как разновидность рекомендательных технологий.

Различие между этими подходами важно и с точки зрения применения в электронной коммерции. Поиск незаменим, когда пользователь точно знает, что хочет найти. Рекомендательная система незаменима, когда пользователь находится в состоянии неопределённости, исследует ассортимент или готов к неожиданным, но релевантным предложениям. Именно в последнем случае рекомендательные системы генерируют наибольшую коммерческую ценность: они продают то, что пользователь не искал, но готов купить.

1.4 Классификация рекомендательных систем

Выделяют три основных класса алгоритмов рекомендательных систем в зависимости от типа данных, на которые они опираются: совместная фильтрация, контентная фильтрация и гибридные системы [3], что наглядно представлено на рисунке 1.



Рисунок 1 – Классификация рекомендательных систем

1.4.1 Коллаборативная фильтрация (Collaborative Filtering)

Основной принцип коллаборативной фильтрации состоит в следующем: если группа пользователей имеет схожие оценки или историю покупок с конкретным пользователем, то товары, понравившиеся этой группе, с высокой вероятностью понравятся и ему. Система ищет пользователей с близкими предпочтениями и рекомендует тот пласт контента, который текущий пользователь ещё не видел, но который есть у его «соседей».

Существуют два варианта подхода [4]:

- user-based: Система ищет других пользователей, чья история оценок или покупок максимально похожа на историю текущего пользователя. Найденная группа «похожих соседей» становится источником рекомендаций: если все они высоко оценили какой-то товар, которого текущий пользователь ещё не видел, система предложит его. Математически сходство между пользователями вычисляется по их строкам в матрице «пользователь–товар» — например, через косинусное расстояние. Интуитивная логика подхода проста: «люди с похожими вкусами, вероятно, оценят одно и то же»;
- item-based: Вместо поиска похожих людей система ищет товары, похожие на те, которые пользователь уже купил или высоко оценил, и рекомендует их. Сходство между товарами также вычисляется по

матрице «пользователь–товар», но уже по столбцам: два товара считаются похожими, если их одинаково оценивает одна и та же группа пользователей.

Выбор между двумя вариантами определяется соотношением числа пользователей и товаров: если товаров больше, чем пользователей — предпочтительна user-based фильтрация; если пользователей больше — item-based. Преимущество коллаборативной фильтрации — в её независимости от содержимого товара: системе не нужно знать, что именно представляет собой рекомендуемый объект. Это позволяет выявлять неочевидные, межкатегорийные связи между товарами.

Однако существуют принципиальные ограничения [5]:

- разреженность матрицы оценок: Большинство пользователей ставят оценки лишь небольшому числу товаров, и найти «соседей» среди тысяч позиций становится затруднительно;
- проблема холодного старта: Для новых пользователей и новых товаров исторических данных ещё нет, и система не может сформировать содержательные рекомендации;
- зависимость качества рекомендаций от объёма накопленных данных: Метод слабо применим для небольших и молодых платформ.

1.4.2 Контентная фильтрация (Content-based Filtering)

Контентная фильтрация работает на основе метаданных объектов каталога. Система формирует профиль пользователя, обобщая характеристики товаров, с которыми он взаимодействовал положительно, и затем подбирает новые объекты с близким вектором признаков. Источниками признаков товара могут служить категория, бренд, ценовой диапазон, текстовое описание и технические характеристики. Главное преимущество подхода — отсутствие зависимости от данных других пользователей: новый товар с подробным описанием может быть рекомендован сразу после добавления в каталог. Кроме того, рекомендации контентного метода легко интерпретируемы. Основной недостаток — склонность к зацикливанию (overspecialization): пользователю предлагаются только объекты, похожие на те, с которыми он уже взаимодействовал, что сужает разнообразие рекомендаций и затрудняет открытие новых категорий. Качество метода также сильно зависит от полноты и точности описания товаров в каталоге [6].

1.4.3 Гибридные системы

Гибридные рекомендательные системы возникли как ответ на ограничения обоих базовых методов. У совместной фильтрации есть требование стабильного потока обратной связи от пользователей, а контентная фильтрация требует подробного описания каждого объекта — гибридный подход позволяет использовать одновременно оба типа данных и компенсировать слабые стороны каждого. Смешанные рекомендаторы возвращают пользователю результаты нескольких систем одновременно — этот подход позволяет при недостатке персональных данных дополнять выдачу наиболее популярными объектами.

В данной работе реализована рекомендательная система, которую по своей архитектуре следует отнести к классу контентной фильтрации. Система не использует данные о поведении других пользователей, их оценках или истории покупок — она опирается исключительно на текстовое содержание отзывов и описаний товаров. Профиль запроса пользователя формируется на основе семантики введённого текста, а рекомендации строятся через сравнение векторного представления запроса с векторными представлениями товаров.

Вместе с тем реализованная система обладает рядом черт, отличающих её от классической контентной фильтрации. Традиционный контентный подход опирается на структурированные атрибуты товаров: категорию, бренд, технические характеристики. В данной работе основным источником информации служат неструктурированные тексты отзывов покупателей, которые кодируются трансформерными языковыми моделями в плотные векторные представления. Это позволяет системе улавливать семантические связи, недоступные при работе с формальными атрибутами: отзыв «мягкая подошва, нога не устаёт» и описание товара «анатомическая стелька, амортизирующая подошва» окажутся семантически близки в векторном пространстве, хотя не имеют общих ключевых слов. В этом смысле система реализует семантическую контентную фильтрацию на основе нейронных представлений текста, что является более современным и гибким вариантом классического подхода.

1.5 Отзывы как источник информации о предпочтениях

Отзыв представляет собой субъективное мнение покупателя о товаре, выраженное в текстовой форме после непосредственного опыта его использования. В отличие от формальных характеристик из карточки товара, отзыв отражает личное восприятие: насколько товар оправдал ожидания, какие свойства оказались важными на практике, что вызвало удовлетворение или разочарование. Субъективность отзыва — одновременно его ограничение и ценность: именно она делает текст информативным с точки зрения предпочтений конкретного человека, а агрегация множества субъективных мнений позволяет получить статистически значимую картину восприятия товара широкой аудиторией.

1.5.1 Отличие от явных оценок

На большинстве маркетплейсов пользователи могут выразить своё отношение к товару в двух формах: поставить числовую оценку (количество звёзд) и написать текстовый отзыв. Звёздный рейтинг удобен для математической обработки, однако его однозначность оказывается ограничением. Оценка «3 звезды» может означать принципиально разные вещи: покупатель доволен качеством, но разочарован доставкой; товар хорош, но не соответствует описанию; продукт устроил в целом, но одна деталь снизила впечатление. Числовая оценка не позволяет различить эти случаи. Крупные маркетплейсы, тратят значительные ресурсы на то, чтобы побудить покупателей не только выставить оценку, но и написать развёрнутый отзыв. Это неслучайно: текстовый отзыв содержит причины оценки и позволяет системе понять не только итоговое отношение покупателя, но и конкретные аспекты товара, которые на него повлияли. Фраза «отличный материал, но швы расходятся через месяц» несёт принципиально больше информации, чем оценка «3 из 5».

Наряду с явными оценками рекомендательные системы широко используют неявные поведенческие сигналы: просмотры страниц, время на странице, добавление в корзину, повторные покупки. Эти данные накапливаются автоматически и являются обильным источником информации. Однако неявные данные страдают принципиальной неопределённостью интерпретации. Просмотр товара может означать

интерес, случайный клик, сравнение с конкурентом или поиск подарка другому человеку. Покупка не обязательно означает удовлетворённость: товар мог оказаться не тем, что ожидалось. Текстовый отзыв лишён этой неопределённости — он представляет собой сознательную, аргументированную оценку пользователя, выраженную после реального опыта использования товара.

Текст отзыва является многоуровневым источником информации. Общая тональность — наиболее очевидный уровень: классификация отзыва как положительного, отрицательного или нейтрального позволяет дополнить или скорректировать числовой рейтинг. На практике тональность и числовая оценка нередко расходятся: пользователь может поставить четыре звезды, написав при этом преимущественно критический текст. Аспектная тональность — более тонкий уровень: отзыв может содержать разные оценки разных свойств товара — качества материала, соответствия размеру, скорости доставки, упаковки, соотношения цены и качества. Эмоциональная окраска отзыва — сила выраженной эмоции, наличие сарказма или иронии — несёт информацию о значимости оценки для пользователя. Наконец, степень доверия к отзыву определяется рядом факторов: подтверждена ли покупка, насколько детален текст, оценили ли другие пользователи отзыв как полезный.

1.5.2 Влияние пользовательских отзывов на работу алгоритмов рекомендаций

Классическая матрица «пользователь–товар» оперирует единственным числом на каждую пару — оценкой или фактом покупки. Интеграция текстовых отзывов позволяет сделать эту матрицу значительно более информативной. Первый способ интеграции — использование тональности отзыва для взвешивания оценок: если пользователь поставил товару четыре звезды, но написал преимущественно критический текст, эффективная оценка в матрице может быть скорректирована вниз. Такая коррекция делает матрицу более точным отражением реальных предпочтений, а не формально выставленных оценок. Второй способ — построение аспектных профилей пользователей на основе упоминаемых в их отзывах свойств товаров. Два пользователя с одинаковыми звёздными рейтингами, но разными аспектными профилями могут иметь принципиально разные вкусы — и учёт этого при вычислении сходства повышает качество рекомендаций.

В контентной фильтрации отзывы выполняют функцию обогащения вектора признаков товара. Формальная карточка товара содержит структурированные атрибуты, заполненные продавцом. Однако покупателей нередко интересуют совсем другие свойства — насколько удобна застёжка, не линяет ли ткань, соответствует ли реальный размер указанному. Эта информация, как правило, отсутствует в официальном описании, но активно обсуждается в отзывах. Анализ отзывов позволяет дополнить вектор его признаков «живыми» характеристиками, отражающими реальный пользовательский опыт. Аналогичным образом анализ написанных пользователем отзывов позволяет построить его динамический профиль предпочтений: не просто список купленных категорий, а описание того, какие именно свойства товаров он ценит.

Наиболее полно потенциал отзывов реализуется в гибридных архитектурах. Для решения проблемы холодного старта нового товара отзывы оказываются незаменимым ресурсом: даже при наличии лишь нескольких первых покупок первые отзывы уже позволяют построить аспектный вектор товара и рекомендовать его пользователям, чей профиль предпочтений совпадает с упомянутыми характеристиками. Это устраняет задержку между появлением товара в каталоге и его включением в рекомендательную систему. Отзывы также повышают точность рекомендаций для малопопулярных товаров, у которых недостаточно поведенческих данных для работы коллаборативной фильтрации. У такого товара может быть всего десяток покупок, но содержательные отзывы дают системе полное представление о его свойствах и позволяют находить для него релевантную аудиторию. Таким образом, отзывы расширяют охват рекомендательной системы, включая в неё товары, которые иначе оставались бы в тени.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

В практической части данной работы рассматривается процесс разработки рекомендательной системы на основе текстов пользовательских отзывов маркетплейса Wildberries. В частности, внимание уделяется выбору языковых моделей для векторизации текстов, реализации трёх методов классификации и поиска товаров, организации хранения данных и вычисленных векторных представлений, а также сравнительному анализу результатов.

Определение подходящей языковой модели для векторизации русскоязычных текстов является одним из ключевых решений при построении семантической рекомендательной системы. Будет проведён сравнительный анализ нескольких моделей — от статических представлений Word2Vec до современных трансформерных моделей RuBERT, MiniLM и E5-large — и на основе качества классификации отзывов по товарным категориям будет обосновано использование трёх финальных моделей.

Важным аспектом практической реализации является организация хранения и обработки данных. Будет рассмотрено подключение к базе данных PostgreSQL, развёрнутой в среде контейнеризации Docker, чанковое чтение данных для работы с большими объёмами отзывов в условиях ограниченных ресурсов, а также система кеширования вычисленных эмбеддингов в формате HDF5, обеспечивающая однократное вычисление векторных представлений и их повторное использование без пересчёта.

Отдельное внимание уделяется решению сопутствующих задач, возникающих при работе с реальными данными маркетплейса: балансировке несбалансированных классов товарных категорий, предобработке русскоязычных текстов отзывов, а также корректному разбиению данных на обучающую, валидационную и тестовую выборки с сохранением пропорций классов.

Большая часть практической работы посвящена реализации и сравнению трёх методов классификации отзывов и поиска товаров: логистической регрессии, центроидного классификатора и сиамской нейронной сети с метрическим обучением. Для каждого метода проводится оценка качества классификации по метрике взвешенной F1-меры. В качестве списка рекомендаций система предлагает пользователю список товаров с

наиболее подходящими характеристиками.

2.1 Подготовка и выбор технологий

На входе система получает текстовый запрос пользователя на естественном языке (например, «тёплые кроссовки для зимы»). Запрос кодируется одной из трёх моделей (RuBERT, MiniLM или E5) в вектор эмбединга. Параллельно все товары из базы Wildberries уже закодированы заранее и хранятся отдельно для каждой модели и каждого метода поиска. Далее работает один из трёх методов: логистическая регрессия, центроидный классификатор, сиамская сеть. На выходе — топ-10 товаров, ранжированных по релевантности запроса.

2.1.1 Выбор языка программирования Python для реализации системы

Выбор языка программирования для реализации рекомендательной системы определяется не только синтаксическими удобствами, но и совокупностью практических факторов: наличием специализированных библиотек, зрелостью экосистемы машинного обучения, поддержкой аппаратного ускорения и возможностью интеграции разнородных компонентов в единый исследовательский конвейер. В настоящей работе в качестве основного инструмента разработки использован ЯП Python. Python занимает доминирующее положение в области машинного обучения и обработки естественного языка, что подтверждается как академической практикой, так и индустриальными стандартами. Все ключевые библиотеки, применяемые в данном исследовании, либо написаны на Python, либо предоставляют Python в качестве основного интерфейса. Помимо этого, ЯП содержит PyTorch — фреймворк глубокого обучения, используемый для реализации сиамской нейронной сети, Sentence Transformers — надстройка, обеспечивающая единый интерфейс для трёх применяемых моделей, scikit-learn — библиотека классического машинного обучения, предоставляющая реализации логистической регрессии и метрик качества, а также инструменты работы с числовыми массивами и табличными данными. Принципиально важно, что все перечисленные инструменты устанавливаются единой командой в одну строку, что невозможно ни для одного

альтернативного языка с сопоставимой полнотой инструментария.

2.1.2 Jupyter Notebook как среда итеративной разработки

Jupyter Notebook — интерактивная среда разработки, представляющая собой веб-приложение, в котором код, результаты его выполнения, визуализации и текстовые пояснения объединены в единый документ. Документ организован в виде последовательности ячеек: каждая ячейка содержит либо исполняемый код, либо разметку на Markdown. Ячейки выполняются независимо и в произвольном порядке, а результаты — числа, таблицы, графики, текстовый вывод — отображаются непосредственно под ячейкой и сохраняются в файле формата `.ipynb`. Для исследовательской разработки рекомендательной системы такая организация принципиально важна. Процесс включает качественно разные этапы: подключение к PostgreSQL и выгрузка данных, предобработка текстов, векторизация через тяжёлые трансформерные модели, обучение классификаторов, оценка метрик, визуализация матриц ошибок, интерактивное тестирование через диалоговый цикл. Каждый из этих этапов занимает разное время и может требовать повторного запуска независимо от остальных. В Jupyter Notebook результат векторизации, сохранённый в памяти ядра, доступен на всех последующих шагах без повторного вычисления — что особенно существенно при работе с моделями RuBERT, MiniLM и E5, каждая из которых кодирует десятки тысяч отзывов и товаров. Среда также обеспечивает воспроизводимость экспериментов: фиксация зерна генератора случайных чисел (`SEED = 42`), последовательная нумерация ячеек и сохранение всех промежуточных выводов позволяют точно восстановить любой шаг разработки. Jupyter notebook позволяет пройти полный цикл от загрузки кэша эмбедингов до сравнительной оценки всех трёх методов классификации и поиска товаров.

2.1.3 Поиск данных и структура выбранных датасетов

Для реализации проекта по загрузке и анализу данных Wildberries потребовалось найти качественные открытые наборы данных, отражающие как структуру товарного каталога, так и пользовательские оценки. В процессе поиска было необходимо определить наиболее подходящие источники, содержащие связанную информацию о товарах и отзывах. Подобные датасеты

часто публикуются большими маркетплейсами и корпорациями в открытом доступе. Один из главных стимулов к публикации датасетов — возможность для других исследователей проводить новые изыскания на основе уже собранных данных. Как отмечают эксперты, очень редко потенциал исследовательского датасета полностью исчерпывается в рамках одного проекта.

При поиске датасетов были изучены несколько ключевых платформ для публикации открытых данных:

- **Kaggle Datasets:** Популярная платформа для поиска датасетов, содержащая более 500 тысяч наборов данных от пользователей и компаний, охватывающих компьютерное зрение, NLP, финансы, здравоохранение, социальные сети и другие области. Платформа предоставляет инструменты для исследования данных и проведения соревнований;
- **GitHub:** Крупнейший репозиторий кода, где также публикуются датасеты в виде файлов или ссылок на внешние хранилища. На GitHub были найдены многочисленные проекты, связанные с анализом данных, включая примеры датасетов товаров и скрипты для парсинга;
- **Hugging Face Datasets:** Одна из ведущих платформ, насчитывающая более 4 тысяч открытых датасетов, охватывающих текст, изображения, аудио и другие модальности. На этой платформе были найдены датасеты с многих крупных интернет магазинов.

Подходящие датасеты были найдены на платформе Hugging Face. Для реализации проекта были выбраны два взаимосвязанных датасета: первый, `wb-feedbacks` [7], содержит информацию об отзывах, второй, `wb-products` [8], содержит информацию о товарах, на которые эти отзывы оставлены. Оба датасета состоят из нескольких файлов ввиду большого размера.

Датасет товаров содержит структурированную информацию о товарах маркетплейса.

На основании представленных данных можно выделить следующие поля:

- `nm_id` — уникальный числовой идентификатор конкретного товарного варианта (артикул товара на уровне цвета или размера);
- `imt_name` — название товарной карточки;
- `nm_colors_names` — Одна из ведущих платформ, насчитывающая

более 4 тысяч открытых датасетов, охватывающих текст, изображения, аудио и другие модальности. На этой платформе были найдены датасеты с многих крупных интернет магазинов;

- `subj_name` — подкатегория товара («Кроссовки», «Конверты для малышей»);
- `subj_root_name` — корневая категория товара («Обувь», «Одежда для малышей»);
- `description` — текстовое описание товара («Летний конверт в набор на выписку новорождённого»).

В данной работе из этого датасета использовались поля `nm_id`, `imt_name`, `subj_root_name`, а также `description`. Поля `imt_name` и `description` объединялись в единый текст по формуле `imt_name + ". " + description` и использовались для построения векторных представлений товаров, формирующих поисковый индекс. Такое объединение обусловлено тем, что название товара содержит его тип и бренд, а описание — детальные характеристики; вместе они дают языковой модели наиболее полное представление о товаре. Поля `nm_colors_names`, `imt_id` и `subj_name` в работе не использовались: цвет не несёт семантической нагрузки для задачи классификации по категориям, идентификатор карточки избыточен при наличии `nm_id`, а подкатегория дублирует информацию корневой категории на более детальном уровне.

Датасет отзывов содержит пользовательские отзывы на товары маркетплейса.

Он включает следующие поля:

- `nmId` — числовой идентификатор товара, на который оставлен отзыв;
- `productValuation` — числовая оценка товара пользователем по шкале от 1 до 5;
- `color` — цвет товара, на который оставлен отзыв;
- `text` — текст отзыва покупателя;
- `answer` — ответ продавца на отзыв.

В работе из этого датасета использовались поля `nmId`, `text` и `productValuation`. Текст отзыва являлся основным источником данных: именно он подавался на вход языковым моделям для построения векторных представлений, на которых обучались классификаторы и строился поисковый индекс. Поле `productValuation` использовалось при формировании среднего

рейтинга товара, отображаемого в результатах поиска. Поля `color` и `answer` в работе не использовались: цвет дублирует информацию из датасета товаров, а ответ продавца не отражает мнение покупателя и не является релевантным для задачи классификации предпочтений.

Ключевым элементом, связывающим оба датасета в единую структуру, является поле `nm_id` — уникальный числовой идентификатор товарного варианта. В датасете товаров это поле называется `nm_id`, в датасете отзывов — `nmId`; оба поля содержат один и тот же идентификатор и служат внешним ключом для объединения таблиц. Связь реализована по принципу «один ко многим»: одному товару соответствует множество отзывов. Это означает, что для одного значения `nm_id` в таблице товаров существует потенциально неограниченное число строк в таблице отзывов — каждая строка представляет отдельный отзыв покупателя на данный товар. На скриншотах это хорошо прослеживается: товар с `nm_id = 5 511 786` имеет множество строк в таблице товаров (по числу цветовых вариантов), а товар с `nmId = 13 205 695` в таблице отзывов представлен десятками строк с разными текстами при одинаковом идентификаторе. Именно это объединение является центральным для всей системы. Без связи по `nm_id` невозможно сопоставить текст отзыва с категорией товара — а это сопоставление является фундаментом обучения всех трёх классификаторов. Отзыв сам по себе не несёт информации о категории товара: текст «Милый домик, прекрасное украшение» не позволяет определить категорию без обращения к карточке товара. Только через объединение по `nm_id` можно узнать, что этот отзыв относится к категории «Декор», и использовать его как обучающий пример. На скриншотах это хорошо прослеживается: товары с `nm_id = 1277064` и `12770649` — см. рисунок 3 — находятся в таблице отзывов (рисунок 2) под теми же `nm_id`. Записей с такими `nm_id` больше одной, значит, на данные товары больше одного отзыва.

	123 id	123 nm_id	123 product_valuation	AZ color	AZ review_text	AZ answer
1	306 532	1 277 064		4 желтый	стопа 24,5 - 38 размер в самый раз. ремешки как бумажные	
2	306 533	1 277 064		5 желтый	С 39 размером взяла 40 - в самый раз. Вовсе не смотрятся	
3	306 526	1 277 064		2 желтый	Единственный плюс этих босоножек - цвет. Идут размер в р	
4	306 527	1 277 064		3 желтый	На свой 39-40 примеряла 40 размер- оказались большие в	
5	306 528	1 277 064		3 желтый	Симпатичные босоножки. Цвет как на фото, не маломерки,	
6	306 529	1 277 064		4 желтый	мне понравились. взяла. хорошо смотрятся на ноге, размер	
7	306 530	1 277 064		4 желтый	оч понравились. но выбор сделала в пользу других из-за ц	
8	306 531	1 277 064		4 желтый	по качеству один минутак как если порвутся,а ремешки не	
9	306 534	1 277 064		5 желтый	Эти босоножки были в центре внимания когда я их одела. L	
10	306 535	1 277 064		5 желтый	отличные сандайки по отличной цене.. на ногу 26,7см 41 р	
11	306 571	12 770 649		1 белый	В блоке что то трещит. А так же расплавился мой провод. у	Корректно оформите возврат по браку в разделе По
12	306 572	12 770 649		1 белый	Зарядное устройство сначала работало хорошо быстро зар	Более 2500 покупателей высоко оценили этот товар
13	306 573	12 770 649		2 белый	Очень лёгкая, значительно легче аналогичных по параметр	Более 2500 покупателей высоко оценили этот товар
14	306 574	12 770 649		5 белый	Блок упакован нормально. Пришел целый. Чуда не случилось	Благодарим за отзыв и высокую оценку! Приятного
15	306 575	12 770 649		5 белый	Зарядное устройство рабочее, все подошло, заряжается тел	Благодарим за обратную связь и оценку. Приятного
16	306 576	12 770 649		5 белый	Купила около года как. Для айфона 10. Родное устройство з	Благодарим за обратную связь и покупку! Приятног
17	306 577	12 770 649		5 белый	Отличный адаптер, ничем не отличается от оригинала, а зн	Благодарим за обратную связь и покупку! Мы полнк
18	306 501	12 770 605		5 прозрачный	Неплохой клей. Плюсы: быстро схватывается (минут через 5	
19	306 502	12 770 605		5 прозрачный	Отлично, хотя и не пользовался. Это точно не подделка! Тю	
20	306 503	12 770 605		5 прозрачный	Отличный клей, после китайского двух-компонентного, про	
21	306 504	12 770 605		5 прозрачный	Пользуюсь этим клеем постоянно, обычно покупаю в ленте	
22	306 505	12 770 605		5 прозрачный	Приклеивала гранитную вазочку к гранитной плите поздней	

Рисунок 2 – Данные из wb-feedbacks

	123 id	123 imt_id	123 nm_id	AZ imt_name	AZ subj_name	AZ subj_root_name	AZ nm_colors_names	AZ description
1	824 671	1 058 392	1 277 064	Сандалии	Сандалии	Обувь	желтый	
2	4 296 724	155 270 943	12 770 649	Зарядное устройство	Зарядные устройства	Смартфоны и аксессуары	белый	Универсальное стационарное зарядн
3	3 738 058	1 348 660	1 691 661	Крем для коррекции п	Кремы	Красота		Крем для любого типа кожи разг
4	3 738 059	1 348 661	1 691 662	Крем для коррекции м	Кремы	Красота		
5	3 738 060	1 348 662	1 691 663	Разглаживающий крем	Кремы	Красота		
6	3 738 061	1 348 663	1 691 664	Ночной крем-скраб Те	Кремы	Красота		
7	3 738 062	1 348 664	1 691 665	Разглаживающая маск	Скрабы	Красота		Маска способствует удалению оро
8	3 738 063	1 348 665	1 691 666	Интенсивный комплекс	Косметические набор	Красота	прозрачный	21-дневный интенсивный компле
9	3 738 064	1 348 666	1 691 667	Омолаж. и восстан. кре	Кремы	Красота	серебристый	
10	3 738 065	1 348 667	1 691 668	Подсушивающий гель	Гели	Красота		
11	3 738 066	1 348 668	1 691 669	Эмульсия для лица, 50	Эмульсии	Красота		Увлажняет и матирует. Тип кожи: ж
12	3 738 067	1 348 669	1 691 670	Тонизирующий гель д	Гели	Красота		
13	3 738 068	134 867	155 213	Кофточка	Джемперы	Одежда	коричневый	
14	3 738 069	1 348 670	1 691 671	Очищающая мицелляр	Мицеллярная вода	Красота		
15	3 738 070	1 348 671	1 691 672	Крем-флюид матирует	Кремы	Красота		
16	3 738 071	1 348 672	1 691 673	Регулирующий крем-ф	Кремы	Красота		
17	3 738 072	1 348 673	1 691 674	Очищающая маска-скр	Кремы	Красота		
18	3 738 073	1 348 674	1 691 675	Гель-флюид против не	Флюиды	Красота	белый	
19	3 738 074	1 348 675	1 691 676	Очищающая крем-пен	Пенки	Красота		
20	3 738 075	1 348 676	1 691 677	Кремовый скраб для т	Скрабы	Красота	прозрачный	Скраб удаляет ороговевшие клетк
21	3 738 076	1 348 677	1 691 678	Увлажняющий бальзам	Бальзамы	Красота	белый	
22	3 738 102	1 348 700	1 691 638	Шариковый дезодоран	Дезодоранты	Красота		Не содержит спирт, подходит для ч

Рисунок 3 – Данные из wb-products

2.1.4 Загрузка данных: установка и настройка Docker, развёртывание PostgreSQL и заполнение базы данных

Начать стоит с описание Docker. Docker — это платформа контейнеризации, позволяющая упаковывать приложение вместе со всеми его зависимостями, конфигурационными файлами и системными библиотеками в изолированный контейнер. Контейнер запускается на любой машине, где установлен Docker, и ведёт себя идентично вне зависимости от операционной системы хоста, установленных пакетов и прочих особенностей окружения. Это принципиально отличает контейнер от виртуальной машины: виртуальная машина эмулирует аппаратное обеспечение целиком и содержит полноценную гостевую операционную систему, тогда как контейнер разделяет ядро хостовой ОС и изолирует только пространство имён процессов,

файловую систему и сетевые интерфейсы. Результат — запуск за секунды вместо минут и накладные расходы в несколько мегабайт вместо нескольких гигабайт.

Ключевым понятием в экосистеме Docker является образ (image) — неизменяемый слепок файловой системы, содержащий операционную среду, приложение и его зависимости. Из образа создаётся контейнер — запущенный экземпляр образа с собственным изолированным процессом. Образы описываются в файле Dockerfile или берутся готовыми из публичного реестра Docker Hub, где размещены официальные образы для большинства популярных баз данных, веб-серверов и языковых сред.

Применение Docker в настоящем проекте обусловлено рядом конкретных преимуществ:

- изоляция сервиса базы данных от хостовой системы: PostgreSQL работает в отдельном пространстве процессов и не конфликтует с другими установленными сервисами;
- воспроизводимость конфигурации: Все параметры сервера (версия PostgreSQL, кодировка, порты, ограничения ресурсов) зафиксированы в файле `docker-compose.yml` и могут быть воспроизведены на любой машине одной командой;
- управление ресурсами: Docker позволяет явно ограничить потребление памяти и процессорного времени контейнером, что критично при работе с большими объёмами данных на локальной машине;
- персистентность данных: Через механизм томов (volumes) данные базы хранятся на диске хоста независимо от жизненного цикла контейнера и сохраняются при его перезапуске или пересоздании.

Для управления контейнером используется инструмент Docker Compose, позволяющий описать все сервисы, их параметры и зависимости в едином YAML-файле. Конфигурация проекта определяет один сервис — `postgres` — на основе официального образа `postgres:15-alpine`. Суффикс `-alpine` означает, что образ построен на базе минималистичного дистрибутива Alpine Linux: его размер составляет около 200 МБ против 400+ МБ у стандартного образа на Debian, что сокращает время первоначальной загрузки.

Параметры конфигурации включают следующее. Переменные окружения задают имя пользователя (`admin`), пароль и название базы данных

(wb_reviews, wb_products), а аргумент инициализации `–encoding=UTF8 –lc-collate=C –lc-ctype=C` обеспечивает корректное хранение и сравнение русскоязычных текстов. Маппинг портов 5433:5432 означает, что стандартный порт PostgreSQL (5432) внутри контейнера проброшен на порт 5433 хостовой машины — это позволяет подключаться к базе из Python-кода по адресу `localhost:5433`, не конфликтуя с возможно установленным локальным PostgreSQL на стандартном порту. Ограничения ресурсов (`memory: 3G, cpus: 2.0`) предотвращают монополизацию ресурсов контейнером при параллельной обработке данных. Именованный том `postgres_data` обеспечивает сохранность данных между перезапусками контейнера. Директива `healthcheck` периодически проверяет готовность сервера командой `pg_isready`, что позволяет Python-приложению дожидаться полной инициализации базы перед началом загрузки. Запуск окружения выполняется единственной командой `docker-compose up -d`, после чего сервис PostgreSQL доступен для подключения.

Перейдем к описанию БД. База данных — это организованное хранилище структурированной информации, позволяющее эффективно записывать, хранить, извлекать и модифицировать данные. Сама по себе база данных — лишь набор файлов на диске; работать с ней позволяет система управления базами данных (СУБД) — программный комплекс, предоставляющий интерфейс для выполнения операций над данными, обеспечивающий их целостность, управление одновременным доступом нескольких клиентов и защиту от потери при сбоях. Реляционные СУБД организуют данные в виде таблиц со строго определёнными столбцами и типами данных. Связи между таблицами устанавливаются через общие ключевые поля. Запросы к данным формулируются на языке SQL (Structured Query Language) — декларативном языке, в котором описывается не алгоритм получения данных, а желаемый результат.

PostgreSQL — СУБД с открытым исходным кодом, разрабатываемая с 1986 года и считающаяся одной из наиболее функционально полных реляционных систем. В контексте текущей работы PostgreSQL предпочтителен перед альтернативами по ряду причин. По сравнению с MySQL PostgreSQL демонстрирует более строгое следование стандарту SQL, лучшую поддержку сложных типов данных (в том числе `Text` для длинных строк отзывов, `BigInteger` для артикулов `Wildberries`) и более высокую

производительность при выполнении аналитических запросов с агрегациями и группировками. По сравнению с SQLite — встраиваемой базой данных, хранящей всё в одном файле, — PostgreSQL обеспечивает полноценное многопользовательское подключение, поддерживает параллельные транзакции без блокировок на уровне всей базы и устойчив к конкурентной записи из нескольких процессов, что критично при многопоточной загрузке данных. Помимо этого, PostgreSQL обеспечивает строгую типизацию схемы, декларативные JOIN-запросы между таблицами отзывов и товаров. Для проекта особенно важна поддержка `bulk_insert_mappings` через SQLAlchemy.

SQLAlchemy — это инструментарий для работы с базами данных на Python, позиционируемый в официальной документации как «The Database Toolkit for Python» (docs.sqlalchemy.org). Библиотека предоставляет два уровня взаимодействия с базой данных, которые можно использовать независимо или совместно. Первый уровень — Core — реализует прямую работу с SQL: построение запросов через Python-выражения, управление подключениями, транзакциями и пулом соединений, а также описание схемы таблиц. Core ориентирован на схему базы данных и построен вокруг принципа неизменяемости — запросы формулируются как команды, а не как изменения состояния объектов [9]. Второй уровень — ORM (Object Relational Mapper) — надстройка над Core, которая позволяет описывать таблицы базы данных в виде обычных Python-классов. ORM транслирует Python-классы в таблицы реляционной базы данных и автоматически преобразует вызовы методов в SQL-запросы. В отличие от Core, ORM ориентирован на предметную область и опирается на изменяемое состояние объектов — вместо явных команд разработчик работает с объектами, изменения которых ORM отслеживает и синхронизирует с базой автоматически [9]. В настоящем проекте SQLAlchemy используется на обоих уровнях. Через ORM описаны модели таблиц `wb_reviews` и `wb_products` — Python-классы, унаследованные от `declarative_base()`, поля которых соответствуют столбцам PostgreSQL с указанием типов (`BigInteger`, `Text`, `Boolean` и др.). Через Core выполняется массовая вставка данных методом `bulk_insert_mappings` — он принимает список обычных словарей и формирует один батч-запрос INSERT, что на порядок эффективнее построчной вставки через ORM-сессию. Управление подключением к PostgreSQL реализовано через `create_engine`, а сессии создаются через фабрику `SessionLocal`. Так PostgreSQL эффективно

принимает пакетные вставки тысяч строк за одну транзакцию, что является ключевым фактором производительности при загрузке миллионов отзывов.

2.2 Реализация загрузки данных

Разработка конвейера загрузки данных велась в среде PyCharm — полнофункциональной IDE для Python от JetBrains. PyCharm обеспечивает удобную работу с многофайловыми проектами, встроенный отладчик, инструменты рефакторинга и интеграцию с системами контроля версий. В контексте данного проекта среда использовалась для написания и отладки четырёх ключевых модулей: `database.py` (схема БД и подключение), `optimized_loader.py` (конвейер загрузки отзывов), `optimized_loader_nd.py` (загрузка карточек товаров) и `checkpoint_manager.py` (система контрольных точек).

2.2.1 Таблицы отзывов, товаров и обработка датасетов

В работе представлено 2 таблицы. Таблица `wb_reviews` хранит отзывы покупателей. Ключевые поля: `nm_id` (артикул товара, тип `BigInteger`) — внешний ключ для связи с таблицей товаров; `review_text` (тип `Text`) — полный текст отзыва; `product_valuation` — числовая оценка от 1 до 5; `word_count` и `text_length` — производные метрики, вычисляемые при загрузке; `has_answer` — булев признак наличия ответа продавца; `file_feedb_num` — номер исходного файла, позволяющий отслеживать происхождение записи. Таблица `wb_products` хранит карточки товаров. Ключевые поля: `nm_id` — артикул, обеспечивающий связь с отзывами; `imt_name` — наименование товара; `subj_name` и `subj_root_name` — подкатегория и корневая категория соответственно (последняя является целевой переменной в задаче классификации); `description` — текстовое описание товара; `brand_name` — бренд. После завершения загрузки на поле `nm_id` таблицы товаров создаётся индекс `idx_wb_products_nm_id`, что радикально ускоряет JOIN-запросы вида «выбрать отзывы с корневой категорией товара» — именно такие запросы используются в ноутбуках для формирования обучающей выборки.

Исходные данные представлены в формате JSONL (JSON Lines) — текстовом формате, в котором каждая строка файла является самостоятельным JSON-объектом. Формат удобен для потоковой обработки: в

отличие от стандартного JSON, где весь файл представляет единый объект или массив и должен быть разобран целиком, JSONL допускает построчное чтение без загрузки всего файла в память. На начальном этапе реализации данные читались последовательно строка за строкой. Для каждой строки выполнялись: парсинг JSON (`json.loads()`), извлечение нужных полей с безопасной обработкой отсутствующих ключей (метод `item.get()`), нормализация типов (функция `safe_int`), подсчёт слов (`count_words`), фильтрация коротких отзывов и формирование записи для вставки в базу. Номер исходного файла извлекался из его имени регулярным выражением (`feedbacks?-(+)`). Записи накапливались в список и вставлялись в PostgreSQL пакетами через `bulk_insert_mappings`.

При загрузке первого файла обнаружилась принципиальная проблема: накопление всех обработанных записей файла в памяти перед вставкой в базу приводит к исчерпанию оперативной памяти. Файлы датасета Wildberries имеют размер от сотен мегабайт до нескольких гигабайт; при загрузке одного такого файла целиком в список Python-объектов объём занятой памяти многократно превышает размер файла на диске — каждый словарь Python несёт значительные накладные расходы на хранение метаданных объекта. На машине с 16 ГБ оперативной памяти попытка удерживать в памяти несколько миллионов записей одновременно приводила к исчерпанию доступного объёма и аварийному завершению процесса.

Решением стал переход к чанковой (блочной) обработке, а также загрузка части от общего объема датасетов — если нет вычислительных ресурсов для обработки такого большого объема данных, то нет необходимости загружать их полностью. Далее, файл читается не целиком, а последовательными блоками фиксированного размера. Генераторная функция `read_file_in_chunks` накапливает строки во временный список до достижения порога (`chunk_size`) и возвращает его, после чего список очищается и заполнение начинается заново. Это позволяет удерживать в памяти одновременно лишь один чанк, тогда как остальные данные остаются на диске. Обработка чанков параллелизована: несколько рабочих процессов обрабатывают разные чанки одновременно, что эффективно задействует все доступные ядра CPU.

Переход к чанковой обработке решил проблему единовременной загрузки всего файла, однако выявил вторичную проблему: обработанные

записи из нескольких чанков накапливались в буфере `insert_batch_data` перед массовой вставкой в базу. При достаточно большом числе параллельно завершившихся чанков буфер вновь мог достигать критического размера. Решение состоит в том, что буфер сбрасывается в базу данных не по завершении всего файла, а по достижении порогового размера: как только `insert_batch_data` накапливает количество записей, превышающее удвоенный размер батча вставки (`config.INSERT_BATCH_SIZE * 2`), немедленно вызывается `fast_insert_batch`, после чего буфер очищается (`insert_batch_data = []`). Таким образом, в памяти никогда не хранится более фиксированного числа записей вне зависимости от размера файла и числа параллельных рабочих процессов. Дополнительно конвейер мониторит загрузку оперативной памяти через библиотеку `psutil`: при превышении 85

Загрузка одного файла датасета занимает несколько часов непрерывной работы. В этих условиях любое внешнее прерывание — отключение питания, нехватка дискового пространства, принудительное завершение процесса — означало бы потерю всего прогресса и необходимость начинать с нуля. Для устранения этого риска реализована система контрольных точек (`checkpoint mechanism`). Класс `CheckpointManager` периодически сохраняет текущее состояние загрузки в JSON-файл `checkpoint.json`. Контрольная точка содержит три ключевых поля: путь к обрабатываемому файлу, номер последней обработанной строки и количество успешно вставленных записей. Сохранение реализовано атомарно: данные сначала записываются во временный файл с суффиксом `.tmp`, после чего временный файл переименовывается в `checkpoint.json` через `os.replace`. Переименование в операционных системах является атомарной операцией ядра — либо новый файл полностью заменяет старый, либо замена не происходит вовсе. Это исключает сценарий, при котором контрольная точка оказывается частично записанной и потому некорректной. При повторном запуске конвейер читает `checkpoint.json`, валидирует его содержимое (проверка наличия обязательных полей и корректности типов) и возобновляет чтение файла с сохранённой строки, пропуская уже обработанные данные.

После завершения загрузки таблицы отзывов аналогичным конвейером загружались карточки товаров из файлов серии `basket-XX.json`. Логика обработки реализована в модуле `optimized_loader_nd.py` и структурно идентична загрузке отзывов: чанковое чтение JSONL, параллельная обработка

в пуле процессов, пакетная вставка через `bulk_insert_mappings`. Ключевое отличие состоит в составе извлекаемых полей — из каждой записи берутся идентификаторы товара, его наименование, иерархия категорий и текстовое описание, которые заполняют таблицу `wb_products`. По завершении загрузки обеих таблиц выполнялась верификация корректности данных. Прямой подсчёт числа строк в таблице `wb_reviews` производится SQL-запросом через SQLAlchemy.

2.2.2 Качественное описание датасетов

Таблица отзывов содержит 34 663 015 записей, таблица товаров — 10 177 029 записей. Связь между таблицами осуществляется по полю `nm_id` — уникальному идентификатору товарного варианта. В таблице отзывов содержится 9 054 352 уникальных значения `nm_id`, в таблице товаров — 10 175 701. Множество товаров, имеющих хотя бы один отзыв и присутствующих в обеих таблицах, составляет 2 006 082 `nm_id`. Таким образом, более восьми миллионов товарных позиций из таблицы `wb_products` не имеют ни одного отзыва в загруженном датасете, а около семи миллионов значений `nm_id` из таблицы отзывов не нашли соответствующей записи в таблице товаров. Это объясняется неполным перекрытием датасетов: каждый из них представляет независимую выгрузку, охватывающую разные временные периоды и части каталога. Для обучения классификаторов и построения поискового индекса использовались только товары, присутствующие в обеих таблицах — то есть те, для которых доступно и описание, и хотя бы один отзыв.

Все записи таблицы `wb_reviews` содержат непустой текст отзыва — NULL-значений в поле `review_text` нет. Среднее число слов в отзыве составляет 36,1, медиана — 30 слов, что наглядно видно на рисунке 4, при этом максимальная длина достигает 1 069 слов. Минимальное значение в датасете равно 21 слову, поскольку при загрузке данных из JSON-файлов применялась фильтрация: отзывы длиной не более 20 слов отбрасывались как неинформативные. Отзывы, не прошедшие фильтр, были отсеяны ещё на этапе загрузки в базу данных. средняя длина в 36 слов позволяет языковой модели извлечь семантически значимый вектор, отражающий конкретные свойства товара — материал, посадку, соответствие описанию, качество упаковки. Именно это делает тексты отзывов ценным источником информации для построения поискового индекса.

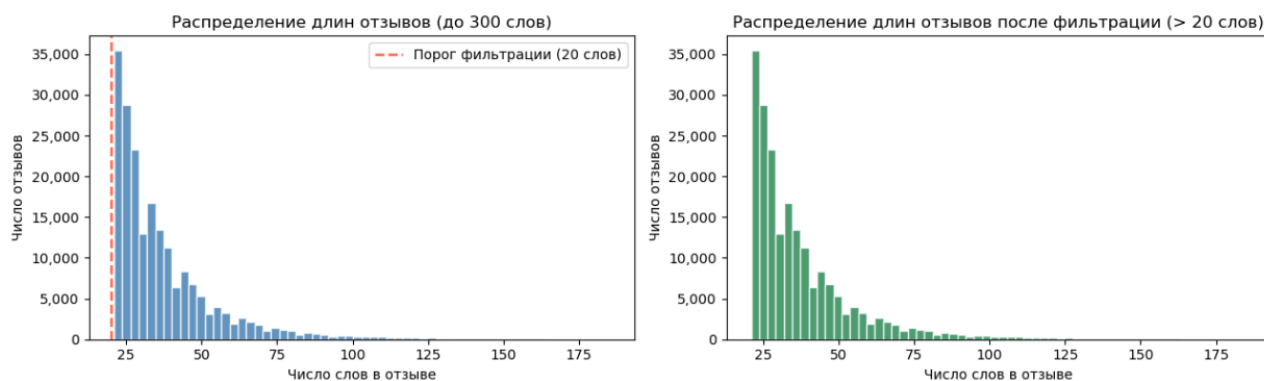


Рисунок 4 – Распределение длин отзывов

Датасет охватывает 65 корневых категорий товаров. Распределение отзывов по категориям крайне неравномерно, что представлено на рисунке 5: самая многочисленная категория — «Одежда» — содержит 3 078 455 отзывов и охватывает 810 475 уникальных товаров. Далее следуют «Обувь» (1 251 488 отзывов, 294 632 товара) и «Красота» (798 540 отзывов, 65 715 товаров). В первую десятку по числу отзывов входят также «Аксессуары», «Бельё», «Игрушки», «Дом», «Посуда и инвентарь», «Книжная продукция и диски» и «Спортивная одежда». Средняя длина отзыва по всем категориям близка и колеблется в диапазоне 34–42 слова, при этом наибольшее среднее число слов демонстрирует категория «Бытовая техника» (42,4 слова) — покупатели этих товаров склонны давать более развёрнутые оценки. Достаточно и очень малочисленных категорий. Самая маленькая из них, «Запчасти для авто», содержит всего 3 отзыва. В процессе подготовки обучающей выборки применялась трёхшаговая балансировка: категории с недостаточным числом отзывов догружались из базы данных, категории, не достигшие минимального порога после догрузки, исключались полностью, а крупные категории обрезались до максимально допустимого размера. Это позволило сформировать сбалансированную обучающую выборку и обеспечить корректное обучение классификаторов.

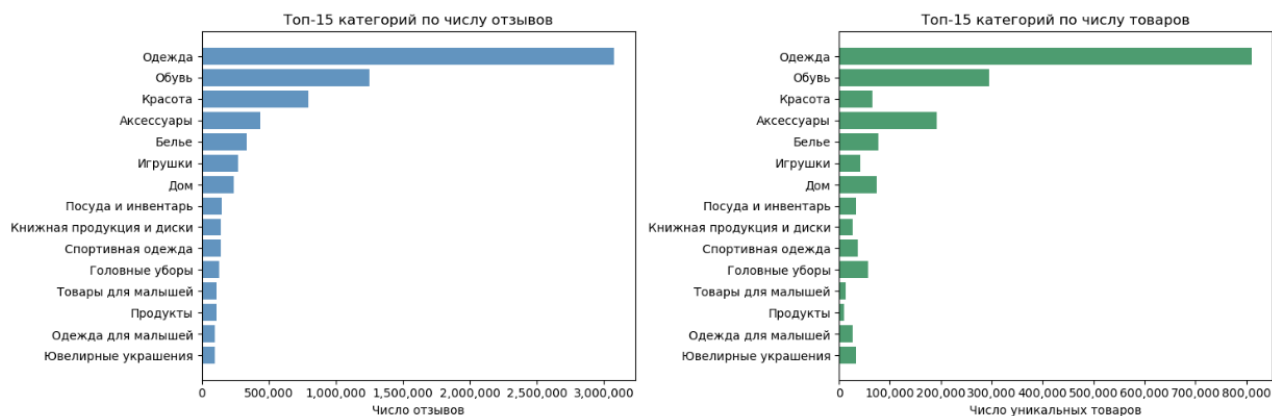


Рисунок 5 – Данные по количеству отзывов

Значительная часть товаров представлена несколькими отзывами от разных покупателей, что отражено на рисунке 6. Это обеспечивает определённую объективность: вектор, построенный на тексте отзыва, отражает не единичное мнение, а характеристики, которые независимо упоминают разные люди — качество материала, соответствие размеру, скорость доставки. Чем больше отзывов на товар, тем полнее его семантический портрет. Наличие нескольких отзывов на товар послужило основой для создания системы. Но многие позиции имеют ровно один отзыв. Для таких товаров поисковый индекс строится на основе единственного текста, который может не отражать полной картины. Именно это обстоятельство послужило одним из оснований для принятого при балансировке решения о догрузке дополнительных отзывов из базы данных для малочисленных категорий: если категория представлена преимущественно товарами с одним-двумя отзывами, её вектор получается менее устойчивым, и увеличение числа обучающих примеров компенсирует эту неравномерность.



Рисунок 6 – Распределение числа отзывов на товар

2.3 Реализация рекомендательной системы

Применение трансформерных языковых моделей к текстам отзывов и описаний товаров порождает числовые векторы — эмбединги — в многомерном пространстве. Сам по себе вектор не несёт практической ценности до тех пор, пока не определено, каким образом сравнивать его с другими векторами. Именно мера близости превращает набор чисел в инструмент поиска: система находит те товары, чьи векторы наиболее близки к вектору запроса пользователя, и возвращает их как рекомендации. Выбор меры близости не является техническим формализмом — он непосредственно влияет на качество рекомендаций. Разные метрики по-разному реагируют на масштаб векторов, их направление и абсолютные значения координат. Для текстовых эмбедингов, где смысл закодирован в направлении вектора, а не в его длине, выбор метрики особенно принципиален.

2.3.1 Меры близости текстовых векторов

Рассмотрим одни из самых распространенных мер для определения сходства текстовых векторов.

Первая из них — косинусное сходство. Оно измеряет угол между двумя векторами в многомерном пространстве. Значение метрики не зависит от длины (нормы) векторов — только от их направления. Именно поэтому мера широко применяется для сравнения текстовых эмбедингов: короткий запрос и длинное описание товара могут быть семантически близки, даже если их векторы существенно различаются по масштабу [10]. Значение метрики лежит в диапазоне $[-1, 1]$: 1 — векторы сонаправлены (тексты максимально близки

по смыслу), 0 — векторы ортогональны (смысловой связи нет), −1 — векторы противоположно направлены.

$$\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}}, \quad (1)$$

где:

- $\vec{a} = (a_1, a_2, \dots, a_n)$ — вектор первого текста (запроса или отзыва) в n -мерном пространстве эмбедингов;
- $\vec{b} = (b_1, b_2, \dots, b_n)$ — вектор второго текста (описания товара или центроида категории);
- $\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$ — скалярное произведение векторов;
- $\|\vec{a}\| = \sqrt{\sum_{i=1}^n a_i^2}$ — евклидова норма (длина) вектора $\vec{a}$;
- $\|\vec{b}\| = \sqrt{\sum_{i=1}^n b_i^2}$ — евклидова норма вектора $\vec{b}$;
- n — размерность пространства эмбедингов (768 для RuBERT, 384 для MiniLM, 1024 для multilingual-E5-large).

Вторая — евклидово расстояние. Евклидово расстояние измеряет длину прямого отрезка между двумя точками в многомерном пространстве. В отличие от косинусного сходства, метрика чувствительна к масштабу векторов: чем длиннее вектор, тем дальше соответствующая точка от начала координат, что искажает сравнение текстов разной длины. На нормализованных до единичной нормы векторах евклидово расстояние и косинусное сходство дают эквивалентное ранжирование [11].

$$d(\vec{a}, \vec{b}) = \|\vec{a} - \vec{b}\| = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}, \quad (2)$$

где:

- $\vec{a} = (a_1, a_2, \dots, a_n)$ — вектор первого объекта;
- $\vec{b} = (b_1, b_2, \dots, b_n)$ — вектор второго объекта;
- $a_i - b_i$ — разность i -х координат двух векторов;
- $(a_i - b_i)^2$ — квадрат разности координат, обеспечивающий неотрицательность и штрафующий большие отклонения сильнее

малых;

- n — размерность пространства эмбедингов;
- $d(\vec{a}, \vec{b}) \in [0, +\infty)$ — значение метрики: 0 соответствует совпадению векторов.

Третья — скалярное произведение. Скалярное произведение суммирует попарные произведения координат двух векторов. На нормализованных до единичной нормы векторах скалярное произведение совпадает с косинусным сходством, поскольку знаменатель формулы (1) обращается в единицу [12]. На ненормализованных векторах метрика зависит как от направления, так и от масштаба, что может смещать результаты ранжирования.

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i, \quad (3)$$

где:

- $\vec{a} = (a_1, a_2, \dots, a_n)$ — вектор первого объекта;
- $\vec{b} = (b_1, b_2, \dots, b_n)$ — вектор второго объекта;
- $a_i b_i$ — произведение i -х координат двух векторов;
- n — размерность пространства эмбедингов;
- $\vec{a} \cdot \vec{b} \in (-\infty, +\infty)$ — значение не ограничено; при нормализованных векторах $\|\vec{a}\| = \|\vec{b}\| = 1$ совпадает с $\cos(\vec{a}, \vec{b})$ из формулы (1).

В классической векторной модели информационного поиска, подробно изложенной в [11], документы и запросы представляются векторами в многомерном пространстве термов. Основной задачей системы является ранжирование документов по их близости к запросу. Авторы обосновывают, что для этой задачи наиболее подходящей мерой является косинусное сходство, поскольку оно нечувствительно к длине документа — ключевому источнику ошибок при сравнении текстов разного объёма.

Принципиальное свойство косинусного расстояния — инвариантность к масштабу вектора: она оценивает только его направление, игнорируя длину. Именно это свойство критично в настоящей работе. Три применяемые модели — RuBERT (размерность 768), MiniLM (384) и multilingual-E5-large (1024) — кодируют тексты разной длины: краткий запрос пользователя («тёплые кроссовки для зимы») и развёрнутое описание товара порождают векторы принципиально разного масштаба. При использовании евклидового расстояния эта разница исказила бы результат: длинный текст оказался бы

«далее» от запроса только из-за масштаба вектора, а не из-за смысловой несхожести. Косинусное сходство нейтрализует этот эффект — два вектора с одинаковым направлением, но разной длиной получают сходство, равное 1. Скалярное произведение же при нормализованных векторах совпадает с косинусным сходством. Таким образом, косинусное сходство выбрано основной мерой для всех трёх моделей и всех трёх методов поиска.

2.3.2 Задача классификации как критерий пригодности модели эмбедингов. Модели Word2Vec, Text2Vec

Прежде чем применять языковую модель для поиска товаров по текстовому запросу, необходимо убедиться, что векторные представления, которые она порождает, несут семантически значимую информацию о предметной области. Сам по себе факт того, что модель является предобученным трансформером, не гарантирует её пригодности для конкретной задачи: модель могла обучаться на корпусах, далёких от тематики маркетплейса, или не поддерживать русскоязычные тексты на необходимом уровне качества. В данной работе в качестве критерия пригодности модели выбрана задача классификации отзывов по корневым категориям товаров. Логика следующая: если эмбединги, порождаемые моделью, несут информацию о категории товара — то есть отзывы на одежду образуют компактные кластеры, отдельные от отзывов на электронику или мебель, — то такие представления пригодны и для поиска схожих товаров. Если же модель не способна разделить отзывы по категориям даже при наличии обучающей выборки, её эмбединги не отражают семантику предметной области и не дадут качественных рекомендаций.

Основной метрикой качества классификации выбрана взвешенная F1-мера (Weighted F1). F1-мера для отдельного класса представляет собой гармоническое среднее точности (Precision) и полноты (Recall):

$$Precision_c = \frac{TP_c}{TP_c + FP_c}, \quad (4)$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}, \quad (5)$$

где:

- TP_c (True Positive) — число объектов класса c , верно отнесённых классификатором к классу c ;
- FP_c (False Positive) — число объектов других классов, ошибочно отнесённых к классу c (ложные срабатывания);
- FN_c (False Negative) — число объектов класса c , ошибочно отнесённых к другим классам (пропуски).

F1-мера для класса c как гармоническое среднее точности и полноты:

$$F1_c = 2 \cdot \frac{Precision_c \cdot Recall_c}{Precision_c + Recall_c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \quad (6)$$

где все обозначения соответствуют формулам (4) и (5).

В дополнение к агрегированной метрике строится матрица ошибок (confusion matrix) — таблица, строки которой соответствуют истинным меткам, а столбцы — предсказанным. Диагональные элементы матрицы показывают число верно классифицированных объектов каждого класса; внедиагональные — систематические ошибки: какие категории модель путает между собой. Анализ матрицы ошибок позволяет выявить семантически близкие категории, которые сложно разделить по тексту отзыва (например, «Аксессуары» и «Аксессуары для волос»), и оценить, носят ли ошибки случайный характер или отражают структурные ограничения модели.

Задача формулируется следующим образом: по тексту — отзыву покупателя или запросу пользователя — определить корневую категорию товара и вернуть топ-10 наиболее релевантных позиций из каталога. Корневая категория — это категория верхнего уровня: «Одежда», «Обувь», «Электроника», «Красота» и так далее. Всего в датасете присутствует 68 таких категорий.

Для численного представления текстов применяются три предобученные модели: RuBERT (размерность вектора 768), MiniLM (384) и multilingual-E5-large (1024). Каждая модель принимает на вход произвольный текст и возвращает плотный вектор фиксированной размерности, кодирующий семантику этого текста. Принципиально важно, что все объекты — отзывы, описания товаров и пользовательские запросы — кодируются одной и той же моделью. Это гарантирует, что все векторы лежат в едином семантическом пространстве и их можно напрямую сравнивать между собой.

Для multilingual-E5-large применяются специальные префиксы: отзывы и описания товаров кодируются с префиксом `passage:`, пользовательские запросы — с префиксом `query:`, что учитывает асимметрию между коротким запросом и длинным документом. Поскольку векторизация десятков миллионов текстов в реальном времени вычислительно нецелесообразна, все эмбединги вычисляются заблаговременно и сохраняются в HDF5-файлах. В момент поиска система загружает нужные матрицы из кэша и выполняет операции над числовыми массивами NumPy, что сводит латентность запроса к миллисекундам. Для каждой из трёх моделей создаются отдельные файлы: эмбединги отзывов (`reviews_embeddings_model.h5`), эмбединги товаров в исходном пространстве (`products_raw_model.h5`) и эмбединги товаров в преобразованном пространстве сиамской сети (`products_siamese_model.h5`).

Прежде чем остановиться на трёх финальных моделях, в ходе исследования были рассмотрены и экспериментально проверены два других подхода к векторизации текстов: статические эмбединги Word2Vec и мультязычная модель `text2vec`.

Word2Vec — нейросетевая модель, предложенная командой Google в 2013 году, обучающая статические векторные представления отдельных слов. Модель обучается на большом корпусе текстов, предсказывая слово по его контексту (вариант CBOW) или контекст по центральному слову (вариант Skip-gram). В результате каждое слово получает фиксированный вектор: слова, регулярно встречающиеся в схожих контекстах, получают близкие векторы. В данной работе для русскоязычных текстов использовалась модель `Word2vec/nlpl_65`, загруженная через Hugging Face Hub с помощью библиотеки Gensim. Модель обучена на корпусе русскоязычных текстов и содержит векторы для слов русского языка. Размерность векторов составляет 300 измерений.

Главная проблема Word2Vec для задачи данной работы состоит в том, что модель оперирует отдельными словами, а не предложениями и не текстами. Чтобы получить вектор целого отзыва, необходимо агрегировать словарные векторы — как правило, усреднением. Такое усреднение уничтожает синтаксическую структуру и контекстуальные отношения между словами: отзыв «отличное качество, плохая доставка» и отзыв «плохое качество, отличная доставка» при усреднении словарных векторов дадут практически идентичные агрегированные представления, поскольку содержат

одни и те же слова. Для задачи классификации по категориям товаров это не критично — оба отзыва, вероятно, относятся к одной категории. Однако для задачи поиска, где требуется точное семантическое сопоставление запроса и описания товара, такая потеря информации принципиальна. Второе ограничение — отсутствие контекстуальности. Word2Vec присваивает каждому слову единственный вектор вне зависимости от контекста. Слово «замок» в отзыве «замок на молнии расстегнулся» и в отзыве «замок двери не закрывается» получит одинаковый вектор, хотя речь идёт о принципиально разных объектах. Для русскоязычных отзывов это особенно значимо из-за богатой морфологии: «куртку», «куртки», «куртке» — разные словоформы, которые Word2Vec может представлять разными векторами или не иметь в словаре вовсе. Третье ограничение — проблема внесловарных токенов (OOV, out-of-vocabulary). Word2Vec строит словарь из корпуса обучения, и любое слово, не вошедшее в этот словарь, не имеет вектора. Отзывы на маркетплейсе содержат опечатки, сокращения, торговые марки, артикулы товаров и разговорные неологизмы, которые с высокой вероятностью отсутствуют в словаре модели.

Что касается `text2vec` — это библиотека и семейство моделей для векторизации предложений, позиционируемое как мультиязычное решение для задач семантического сходства и поиска. Модель `shibing624/text2vec-base-multilingual` основана на архитектуре трансформера и, в отличие от Word2Vec, кодирует текст целиком, а не отдельные слова. Поддержка нескольких языков делала её теоретически применимым кандидатом для работы с русскоязычными текстами.

Модель была загружена и протестирована на парах русскоязычных предложений с явно различающейся семантикой. В ноутбуке `text2vec_multilangcomments.ipynb` было проверено косинусное сходство для двух типов пар: семантически близких предложений и семантически далёких. Результаты оказались неудовлетворительными: модель выдавала высокое косинусное сходство для предложений с противоположным смыслом и не разделяла пары, которые человек однозначно счёл бы несвязанными. Одна из проверочных пар: «Как сменить пароль» и «Где ближайший супермаркет» — тексты, не имеющие ничего общего ни тематически, ни лексически, получили высокое сходство.

Анализ поведения модели позволил выдвинуть объяснение,

зафиксированное непосредственно в ноутбуке: модель, по всей видимости, обучалась преимущественно на китайскоязычных данных, а мультиязычность является скорее декларируемой, чем фактически реализованной. Для русскоязычных текстов модель не имела достаточного обучающего покрытия, вследствие чего её векторные представления не несли содержательной семантической информации о русском тексте. Модель оказалась неспособна разделить семантически далёкие русскоязычные предложения, что делает её непригодной как для классификации отзывов по категориям, так и для поиска товаров по запросу. Этот опыт конкретизировал требования к финальным моделям. Необходима специализация на русском языке (RuBERT). RuBERT представляет собой адаптацию многоязычной модели BERT для русского языка: авторы заменили подсловарь токенов на монолингвальный русский и дообучили модель на корпусе Википедии и новостных текстах [13]. Либо подтверждённое качественное мультиязычное покрытие (MiniLM, multilingual-E5-large). Также, Для multilingual-E5-large применяются специальные префиксы: отзывы и описания товаров кодируются с префиксом `passage`, пользовательские запросы — с префиксом `query`. Это позволит учесть асимметрию между коротким запросом пользователя и длинным текстом описания характеристик товара [14].

2.4 Методы классификации отзывов и поиска товаров

Перейдем к рассмотрению непосредственно методов классификации. Их принципы также лежат в основе проектирования методов поиска.

2.4.1 Метод А — логистическая регрессия

При обучении классификатора на вход подаются векторы отзывов покупателей: текст каждого отзыва кодируется моделью в вектор, и эти векторы с соответствующими метками категорий образуют обучающую выборку. Категория при этом представлена не текстом, а числом-меткой: «Обувь» — это число 1, «Одежда» — число 2 и так далее. При поиске товаров каждый товар представлен вектором, построенным по конкатенации трёх текстовых полей: `imt_name + '. ' + review_text + '. ' + description` — то есть названия товара, одного из его отзывов и официального описания из таблицы `wb_products`. Такой подход принципиально отличается от использования

только карточки товара. Официальное описание фиксирует формальные характеристики, заполненные продавцом: состав, размерный ряд, технические параметры. Отзыв же отражает реальный пользовательский опыт: как сидит, насколько тёплый, соответствует ли размер. Объединение этих двух источников в одном векторе даёт более полное семантическое представление товара — такое, которое одновременно охватывает и формальные характеристики, и живой язык покупателей. Именно последнее критично для поиска: запрос пользователя «мягкие кроссовки для бега» значительно ближе к языку отзыва «очень мягкая подошва, удобно бегать», чем к официальному описанию «верх из сетчатого материала, резиновая подошва». Запрос пользователя кодируется той же моделью в вектор того же пространства.

Логистическая регрессия — линейная модель: она ищет разделяющие гиперплоскости в многомерном пространстве эмбедингов. Для каждого класса модель подбирает вектор весов той же размерности, что и эмбединг. Скалярное произведение весового вектора класса на эмбединг отзыва даёт «балл уверенности» в том, что отзыв принадлежит этому классу. Суть обучения сводится к тому, что модель настраивает веса так, чтобы для правильного класса балл был максимальным: «если в эмбединге активны такие-то измерения — это, вероятнее всего, Одежда; если другие — Электроника». Оптимизация выполняется методом максимального правдоподобия с L2-регуляризацией, предотвращающей переобучение на больших размерностях входного вектора.

Поступает новый отзыв или запрос. Модель кодирует его текст в вектор. Логистическая регрессия умножает этот вектор на матрицу весов и получает по одному числу на каждый класс. Через функцию `softmax` числа превращаются в вероятности. Класс с наибольшей вероятностью — предсказанная категория. Операция занимает миллисекунды, поскольку сводится к одному матрично-векторному умножению.

Запрос пользователя — например, «белые кроссовки 42 размера» — кодируется моделью в вектор. Этот вектор сравнивается косинусным сходством со всеми товарными векторами из файла `products_raw_model.h5`. Файл содержит матрицу $M \times d$, где M — число товаров в каталоге, d — размерность пространства. Для ускорения вычислений все векторы предварительно нормализуются до единичной длины, после чего косинусное сходство сводится к скалярному произведению: вычисляется произведение

матрицы товарных векторов на вектор запроса, результатом является вектор из M чисел — по одному на каждый товар. Числа упорядочиваются по убыванию, и первые 10 позиций возвращаются как рекомендации. Поскольку вектор запроса и товарные векторы построены одной моделью, они лежат в одном пространстве — прямое сравнение корректно.

Ключевая формула метода А — предсказание класса через softmax:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} \frac{\exp(\vec{w}_c \cdot \vec{x} + b_c)}{\sum_{j=1}^C \exp(\vec{w}_j \cdot \vec{x} + b_j)}, \quad (7)$$

где:

- $\vec{x} \in \mathbb{R}^d$ — эмбединг входного текста (отзыва или запроса пользователя);
- $\vec{w}_c \in \mathbb{R}^d$ — вектор обученных весов класса c , d — размерность пространства эмбедингов;
- $b_c \in \mathbb{R}$ — скаляр смещения (bias) класса c ;
- C — общее число классов (корневых категорий товаров);
- $\mathcal{C}$ — множество всех классов;
- $\hat{c}$ — предсказанная корневая категория товара.

2.4.2 Метод Б — центроидный классификатор

Центроид категории строится из отзывов покупателей — это среднее арифметическое векторов всех обучающих отзывов данной категории. Если категория «Обувь» представлена пятьюстами отзывами, её центроид — это среднее пятисот векторов по каждому из d измерений. Центроид отражает не официальное описание категории, а обобщённый голос покупателей: если большинство обувных отзывов упоминают размер, материал и подошву, всё это в усреднённом виде присутствует в центроиде.

Векторы товаров строятся по той же тройной конкатенации `imt_name + '. ' + review_text + '. ' + description`, что и в методе А. Это обеспечивает семантическую совместимость: центроид, построенный из отзывов, сравнивается с товарными векторами, которые тоже содержат отзыв как часть своего представления. Язык, которым описан товар в индексе, совпадает с языком, по которому строился эталон категории.

Для каждого из 68 классов из обучающей выборки отбираются все

отзывы с соответствующей меткой. Их векторы суммируются поэлементно и делятся на количество отзывов. Результат записывается в строку матрицы центроидов размером $C \times d$, где C — число классов. Матрица вычисляется один раз и сохраняется в памяти. Никакого обучения с градиентным спуском не происходит: центроиды — это чисто аналитическая операция усреднения. Если облако точек класса имеет сложную форму с несколькими кластерами, единственный центроид может оказаться в области, далёкой от любого реального отзыва — это структурное ограничение метода.

Поступает новый отзыв. Модель кодирует его в вектор. Вычисляется косинусное сходство этого вектора со всеми 68 центроидами — то есть один вектор сравнивается с матрицей $68 \times d$. Класс, чей центроид оказался ближе всего, — предсказанная категория. Модель отвечает на вопрос: «На отзывы какой категории этот текст похож больше всего?»

Поиск в методе Б двухэтапный, и категория определяется автоматически без участия пользователя. На первом этапе запрос пользователя кодируется моделью в вектор. Этот вектор сравнивается с матрицей центроидов: вычисляется косинусное сходство со всеми 68 центроидами, и выбирается ближайший. Логика такова: если запрос семантически близок к «собирательному образу» обувных отзывов — значит это запрос про обувь. На втором этапе, когда категория определена, вектор запроса сравнивается косинусным сходством с товарными векторами из `products_raw_model.h5`, принадлежащими найденной категории. Перебор ведётся только по товарам этой категории, а не по всему каталогу. Из оставшихся позиций возвращаются топ-10 с наибольшим сходством.

$$\vec{\mu}_c = \frac{1}{|S_c|} \sum_{k \in S_c} \vec{x}_k, \quad \hat{c} = \arg \max_{c \in C} \cos(\vec{x}, \vec{\mu}_c), \quad (8)$$

где:

- $\vec{\mu}_c \in \mathbb{R}^d$ — центроид класса c , геометрический центр облака эмбедингов всех отзывов этого класса в обучающей выборке;
- S_c — множество индексов обучающих отзывов класса c ;
- $|S_c|$ — число отзывов класса c в обучающей выборке;
- $\vec{x}_k \in \mathbb{R}^d$ — эмбединг k -го отзыва;
- $\vec{x} \in \mathbb{R}^d$ — эмбединг входного текста (отзыва или запроса пользователя);

- $\cos(\vec{x}, \vec{\mu}_c)$ — косинусное сходство между вектором входного текста и центроидом класса c ;
- $\mathcal{C}$ — множество всех классов;
- $\hat{c}$ — предсказанная корневая категория товара.

2.4.3 Метод В — сиамская нейронная сеть

Методы А и Б работают с «сырыми» эмбедингами модели — пространство, в котором происходит сравнение, является тем, которое выдаёт предобученный трансформер. Сиамская сеть идёт дальше: она обучает дополнительное нелинейное преобразование, которое переводит все векторы в новое метрическое пространство, специально заточенное под задачу. В этом новом пространстве пары (отзыв, его категория) должны быть геометрически близко, а пары (отзыв, чужая категория) — далеко.

Сиамская сеть работает с парами текстов. Первый элемент каждой пары — текст отзыва покупателя. Второй элемент — текстовое название категории: само слово «Обувь» или «Электроника» подаётся в модель как отдельный текст. Это принципиально отличает метод В от метода Б: там категория представлена усреднёнными отзывами (построена снизу вверх), здесь — своим официальным названием (задана сверху вниз). Сеть учится деформировать пространство так, чтобы эмбединг слова «Обувь» оказался рядом с эмбедингами всех обувных отзывов.

Товарные векторы для файла `products_siamese_model.h5` строятся по той же тройной конкатенации `int_name + '. ' + review_text + '. ' + description` — и затем прогоняются через энкодер сиамской сети. Включение отзыва в представление товара здесь особенно важно: энкодер обучался сближать отзывы с названиями категорий, поэтому товарный вектор, содержащий реальный отзыв, лучше позиционируется в обученном метрическом пространстве, чем вектор, построенный только по официальному описанию.

Энкодер представляет собой двухслойную полносвязную сеть, отображающую исходное пространство эмбедингов в новое пространство размерности 512 вне зависимости от исходной размерности модели. Первый слой сжимает вектор размерности d до 512 измерений, функция активации ReLU вводит нелинейность, второй слой сохраняет размерность 512. Один и тот же энкодер применяется к обоим элементам пары — отзыву и названию категории — что и делает сеть «сиамской»: два идентичных близнеца,

обрабатывающих разные тексты в одном пространстве.

Голова сети принимает четыре представления:

- а) вектор отзыва после энкодера $\mathbf{h}_a$;
- б) вектор категории после энкодера $\mathbf{h}_b$;
- в) поэлементную разность $|\mathbf{h}_a - \mathbf{h}_b|$;
- г) поэлементное произведение $\mathbf{h}_a \odot \mathbf{h}_b$.

Конкатенация этих четырёх векторов даёт представление размерности $512 \times 4 = 2048$, из которого голова извлекает один скаляр — балл принадлежности пары.

Для каждого отзыва формируются две пары: положительная — отзыв и название его истинной категории с меткой 1, и отрицательная — тот же отзыв и название случайно выбранной чужой категории с меткой 0. Функция потерь BCEWithLogitsLoss заставляет сеть давать высокий балл положительным парам и низкий — отрицательным. В процессе обучения энкодер постепенно деформирует исходное пространство: он «растягивает» и «сжимает» координаты так, чтобы отзыв про обувь оказался рядом с вектором слова «Обувь» и далеко от вектора слова «Электроника». Обновление весов выполняется методом Adam через механизм автоматического дифференцирования PyTorch.

Поступает новый отзыв. Модель-трансформер кодирует его в вектор размерности d . Энкодер сиамской сети переводит этот вектор в пространство 512 измерений. Параллельно энкодер переводит в то же пространство эмбединги всех 68 названий категорий. Голова сети вычисляет балл для каждой пары (отзыв, категория). Категория с максимальным баллом — ответ. Ключевое отличие от методов А и Б: сравниваются не исходные эмбединги, а преобразованные — в обученном метрическом пространстве.

Нельзя сравнивать вектор запроса, прошедший через энкодер сиамской сети (512 измерений, обученное пространство), с товарными векторами из `products_raw_model.h5` (исходная размерность, исходное пространство) — это разные пространства, и косинусное сходство между ними не имеет смысла. Поэтому заблаговременно создаётся отдельный файл `products_siamese_model.h5`: тексты товаров, сформированные по схеме `imt_name + '. ' + review_text + '. ' + description`, сначала кодируются трансформером, затем прогоняются через обученный энкодер и сохраняются в новом 512-мерном пространстве. При поиске запрос пользователя проходит

следующий путь: текст запроса поступает в модель-трансформер, которая возвращает вектор размерности d ; этот вектор подаётся в энкодер сиамской сети и преобразуется в вектор размерности 512. Затем вычисляется косинусное сходство этого вектора со всей матрицей из `products_siamese_model.h5` — перебор ведётся по всему каталогу без ограничения категорией. Числа упорядочиваются по убыванию, первые 10 позиций возвращаются как рекомендации. Это потенциально позволяет находить кросс-категорийные рекомендации, выходящие за жёсткие границы отдельных категорий.

$$\mathcal{L} = - \left[y \cdot \log \sigma(s(\vec{h}_a, \vec{h}_b)) + (1 - y) \cdot \log(1 - \sigma(s(\vec{h}_a, \vec{h}_b))) \right], \quad (9)$$

где:

- $\mathcal{L}$ — значение функции потерь на одной обучающей паре;
- $y \in \{0, 1\}$ — метка пары: 1 для положительной пары (отзыв и его истинная категория), 0 для отрицательной (отзыв и чужая категория);
- $\vec{h}_a = \text{enc}(\vec{x}) \in \mathbb{R}^{512}$ — вектор отзыва после прохождения через энкодер сиамской сети;
- $\vec{h}_b = \text{enc}(\vec{e}_c) \in \mathbb{R}^{512}$ — вектор названия категории c после прохождения через тот же энкодер; $\vec{e}_c$ — эмбединг текстового названия категории, полученный трансформером;
- $s(\vec{h}_a, \vec{h}_b) \in \mathbb{R}$ — скалярный балл принадлежности пары, вычисляемый головой сети по конкатенации $[\vec{h}_a; \vec{h}_b; |\vec{h}_a - \vec{h}_b|; \vec{h}_a \odot \vec{h}_b]$;
- $\sigma(z) = (1 + e^{-z})^{-1}$ — сигмоидная функция, преобразующая балл в вероятность принадлежности к положительному классу;
- $\text{enc}(\cdot)$ — энкодер сиамской сети: двухслойная полносвязная сеть $\mathbb{R}^d \rightarrow \mathbb{R}^{512}$ с активацией ReLU.

2.5 Разработка графического интерфейса

Для практической демонстрации работы рекомендательной системы был разработан веб-интерфейс на базе библиотеки Gradio, позволяющий взаимодействовать с системой без запуска Jupyter Notebook. Gradio представляет собой Python-библиотеку с открытым исходным кодом, предназначенную для быстрого создания интерактивных веб-приложений

поверх моделей машинного обучения. Библиотека обеспечивает автоматическое построение пользовательского интерфейса из Python-кода и запуск локального веб-сервера, доступного в браузере по адресу `http://localhost:7860`. Ключевым преимуществом Gradio для целей данной работы является отсутствие необходимости в разработке клиентской части на JavaScript или HTML: весь интерфейс описывается декларативно средствами Python, что позволяет сосредоточиться на логике системы, а не на деталях фронтенд-разработки.

Архитектура приложения организована в виде трёх модулей. Файл `config.py` содержит все пути к HDF5-файлам эмбедингов, пути к сохранённым моделям, названия языковых моделей и прочие константы. Файл `backend.py` реализует логику загрузки моделей, построения поискового индекса и выполнения классификации и поиска. Файл `app.py` отвечает непосредственно за построение и запуск графического интерфейса с использованием Gradio. Подобное разделение обусловлено практическими соображениями. Загрузка языковых моделей семейства BERT и чтение HDF5-файлов с эмбедингами представляют собой ресурсоёмкие операции, которые нецелесообразно повторять при каждом обращении к интерфейсу. В `backend.py` реализован механизм ленивой загрузки с кешированием: все тяжёлые объекты — модели `SentenceTransformer`, матрицы эмбедингов товаров, обученные классификаторы и центроиды категорий — загружаются однократно при первом обращении и сохраняются в словарях-кешах. Последующие запросы из интерфейса используют уже загруженные объекты, что сводит латентность ответа к миллисекундам. Интерфейс реализован с использованием компонента `gr.Blocks`, предоставляющего полный контроль над расположением элементов. Макет разделён на две колонки. В левой колонке размещены настройки системы: выбор задачи, выбор языковой модели и выбор метода работы. В правой колонке расположено текстовое поле для ввода запроса, кнопка запуска и поле вывода результатов.

Выбор задачи осуществляется с помощью группы радиокнопок с двумя вариантами: «Поиск товаров» и «Классификация отзыва». При выборе задачи поиска система принимает произвольный текстовый запрос на русском языке и возвращает ранжированный список из десяти товаров, семантически наиболее близких к введённому тексту. При выборе задачи классификации система принимает текст отзыва покупателя и определяет корневую товарную

категорию, к которой он относится, с указанием уверенности модели в предсказании. Выбор языковой модели также реализован через группу радиокнопок с тремя вариантами: MiniLM, RuBERT и E5-large. Под кнопками отображается краткое описание выбранной модели — её полное название, размерность векторного пространства и характеристика. При смене модели описание обновляется автоматически через обработчик события `model_radio.change`. Там же в левой колонке выводится блок статуса файлов: для выбранной модели отображается наличие или отсутствие каждого из необходимых файлов — HDF5-файлов эмбеддингов, сохранённых классификаторов и весов сиамской сети. Наличие файла обозначается (ok), отсутствие — (no). Это позволяет пользователю сразу видеть, какие компоненты готовы к работе. Выбор метода работы осуществляется третьей группой радиокнопок с тремя вариантами: метод А (логистическая регрессия), метод Б (центроидный классификатор) и метод В (сиамская нейронная сеть). Под кнопками отображается описание принципа работы выбранного метода. Принципиальное отличие методов Б, В от метода А состоит в том, что категория товара при поиске определяется системой автоматически — пользователю не требуется указывать её явно.

В правой части интерфейса расположено многострочное текстовое поле для ввода запроса или текста отзыва. Запуск выполняется либо нажатием кнопки «Запустить», либо клавишей Enter непосредственно в поле ввода — оба действия обрабатываются одной и той же функцией `run`, которая в зависимости от выбранной задачи вызывает соответствующий метод `backend.search` или `backend.classify`. Поле вывода результатов реализовано как не редактируемое текстовое поле с моноширинным шрифтом. При выполнении поиска в поле вывода отображается время выполнения запроса в миллисекундах, число найденных товаров и для каждого товара — его порядковый номер в выдаче, название, артикул, средний рейтинг на основе оценок покупателей и значение косинусного сходства между вектором запроса и вектором товара. Товары упорядочены по убыванию косинусного сходства. При выполнении классификации выводится предсказанная категория, уверенность модели в виде числа от 0 до 1 с визуальной шкалой из блоковых символов и название использованного метода.

В нижней части интерфейса расположен блок примеров запросов, реализованный через компонент `gr.Examples`. Таблица содержит пять готовых

примеров, охватывающих оба типа задач и различные комбинации моделей и методов. Щелчок по строке примера автоматически заполняет все поля формы, что позволяет быстро продемонстрировать работу системы без ручного ввода. При первом обращении к модели, которая ещё не была загружена, система загружает её с Hugging Face Hub или из локального кеша. Для модели MiniLM это занимает несколько секунд, для E5-large — до нескольких минут при первой загрузке из сети ввиду размера модели около одного гигабайта. Последующие запросы с использованием уже загруженной модели выполняются значительно быстрее. В случае отсутствия необходимых файлов система не завершает работу с ошибкой, а отображает в поле вывода информативное сообщение с указанием недостающего файла и инструкцией по его получению. Запуск приложения осуществляется командой `python app.py` из корневой директории проекта. Браузер открывается автоматически благодаря параметру `inbrowser=True`. Параметр `server_name="0.0.0.0"` обеспечивает доступность интерфейса не только с локальной машины, но и с других устройств в одной сети.

Полная схема архитектуры рекомендательной системы приведена на рисунке 7.

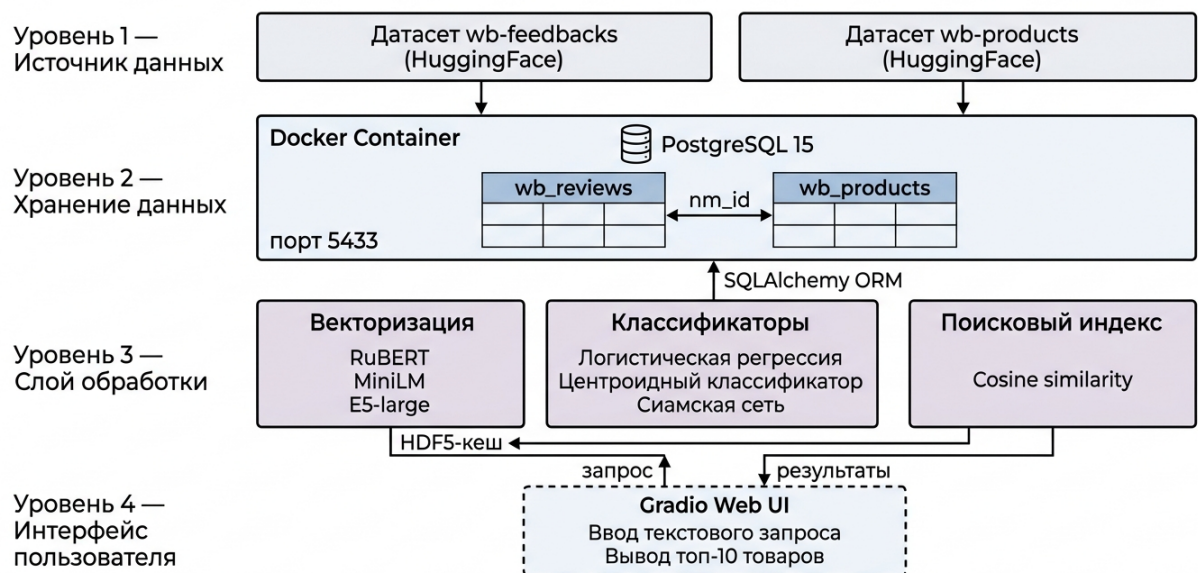


Рисунок 7 – Схема архитектуры рекомендательной системы

2.6 Результаты решения задачи классификации отзывов и поиска релевантных товаров

Далее представлены результаты систематических экспериментов по оценке качества классификации товарных категорий на основе отзывов покупателей. Эксперименты охватывают три модели векторных представлений (RuBERT, MiniLM, multilingual-E5-large), три метода классификации и шесть режимов формирования обучающей выборки — от полностью несбалансированного набора до 100 000 отзывов на категорию. Метрика — F1-мера (weighted), измеренная отдельно на валидационной и тестовой выборках, а также анализ релевантного поиска.

2.6.1 Результаты поиска релевантных товаров

По результатам разработки система способна принимать произвольный текстовый запрос на русском языке и возвращать ранжированный список из десяти товаров, семантически наиболее близких к введённому тексту. Поиск реализован тремя методами для каждой из трёх языковых моделей, что в совокупности даёт девять вариантов работы системы, доступных для сравнения. Для каждого найденного товара система возвращает следующую информацию: артикул товара (nm_id), название товарной позиции, средний рейтинг на основе оценок покупателей и значение косинусного сходства между вектором запроса и вектором товара. Значение косинусного сходства варьируется от 0 до 1 и отражает степень семантической близости запроса к тексту, на основе которого был построен вектор товара. Товары в выдаче упорядочены по убыванию этого значения — наиболее релевантный результат занимает первую позицию. Поисковый индекс в финальной версии системы построен по тексту, объединяющему название товара, один из отзывов покупателей и описание продавца. Такой подход позволяет улавливать как формальные характеристики товара из карточки продавца, так и живой потребительский опыт из отзыва, что расширяет семантическое покрытие индекса по сравнению с векторизацией только официального описания. В результате система способна находить товары не только по точным атрибутам, но и по косвенным признакам, упоминаемым покупателями. Три реализованных метода поиска различаются как пространством, в котором выполняется поиск, так и механизмом определения релевантных товаров.

Методы А и Б используют единый индекс в пространстве выбранной языковой модели, тогда как метод В применяет отдельный индекс, построенный в обученном метрическом пространстве сиамской сети. Принципиальное отличие метода Б состоит в том, что категория товара при поиске определяется автоматически на основе семантического сравнения запроса с центроидами категорий — пользователю не требуется указывать её вручную. В методе А категория задаётся пользователем через выпадающий список и выступает как фильтр, сужающий пространство поиска до товаров выбранного раздела каталога. Метод В выполняет поиск по всему каталогу без ограничения категорией, что потенциально позволяет находить кросс-категорийные результаты. Для практической демонстрации работы системы реализован веб-интерфейс на базе библиотеки Gradio, позволяющий работать с системой без запуска Jupyter Notebook. Интерфейс предоставляет возможность выбора задачи — классификация отзыва или поиск товаров, — выбора языковой модели из трёх доступных (RuBERT, MiniLM, E5-large) и выбора метода (А, Б или В). Пример работы системы в режиме поиска товаров представлен на рисунке 8, в режиме классификации отзыва — на рисунке 9

Рекомендательная система

Семантический поиск товаров и классификация отзывов по текстовому запросу

Настройки

Задача

Что делает система с вашим текстом

Поиск товаров

Классификация отзыва

Модель векторизации

MiniLM

RuBERT

E5-large

DeepPavlov/rubert-base-cased-sentence | dim=768 | специализирован на русском

Метод

A (Лог. регрессия)

Б (Центроугры)

В (Сиамская сеть)

Эмбеддинги → LogReg. Для поиска: прямое косинусное сравнение с индексом товаров.

Категория товаров

Выбор категории для метода А.

Одежда

Запрос

куртка универсальная удлиненная зимняя

Запустить

Результат

Отличный вариант для повсе...

6. Пальто

Артикул: 730213

Рейтинг: 5.00 ★★★★★ | Сходство: 0.8039

Превосходное пальто насыщенного цвета. Модель с воротником-стойкой, капюшоном и длинными рукавами имеет двубортную застежку на пуговицы. Все швы оформ...

7. Куртка

Артикул: 1676281

Рейтинг: 3.00 ★★★☆☆ | Сходство: 0.7993

Куртка зимняя

8. Куртка

Артикул: 1676281

Рейтинг: 4.00 ★★★★★ | Сходство: 0.7826

Куртка зимняя

Статус файлов для RuBERT: (ok) Файл 2: эмбеддинги товаров (raw) (ok) Файл 3: эмбеддинги товаров (siamese) (ok) Файл 1: эмбеддинги отзывов (ok) Модель: LogReg (joblib) (ok) LabelEncoder (joblib) (ok) Сиамская сеть (.pt)

Примеры запросов

Нажмите на строку, чтобы заполнить форму

Задача	Модель векторизации	Метод	Текст запроса / отзыва	Категор
Поиск товаров	MiniLM	A (Лог. регрессия)	тёплые кроссовки для зимы	
Поиск товаров	RuBERT	Б (Центроугры)	мягкая подошва, не устают ноги	
Поиск товаров	E5-large	В (Сиамская сеть)	крем для сухой кожи рук увлажняющий	
Классификация отзыва	MiniLM	A (Лог. регрессия)	Ткань приятная, не мнётся, цвет яркий	
Классификация отзыва	RuBERT	В (Сиамская сеть)	Кастрюля отличная, дно толстое	

Использовать через API · Создано с помощью Gradio · Настройки

Рисунок 8 – Интерфейс системы в режиме поиска товаров

54

Рекомендательная система

Семантический поиск товаров и классификация отзывов по текстовому запросу

Настройки

Задача

Что делает система с вашим текстом

☐ Поиск товаров

☒ Классификация отзыва

Модель векторизации

☒ MiniLM

☐ RuBERT

☐ E5-large

paraphrase-multilingual-MiniLM-L12-v2 | dim=384 | быстрая

Метод

☒ А (Лог. регрессия)

☐ Б (Центроугры)

☐ В (Сиамская сеть)

Эмбеддинги → LogReg. Для поиска: прямое косинусное сравнение с индексом товаров.

Запрос

для неспешных прогулок подойдут, на ногу сели хорошо качество чувствуется, швы прочные, шнурки в подарок, очень приятно, думаю, буду долго носить

Запустить

Результат

Время: 13014.1 мс

Категория: **Обувь**

Уверенность: 99.89%

Метод: А (Лог. регрессия)

Статус файлов для MiniLM: (ok) Файл 2: эмбеддинги товаров (raw) (ok) Файл 3: эмбеддинги товаров (siamese) (ok) Файл 1: эмбеддинги отзывов (ok) Модель: LogReg (joblib) (ok) LabelEncoder (joblib) (ok) Сиамская сеть (.pt)

Примеры запросов

Нажмите на строку, чтобы заполнить форму

Задача	Модель векторизации	Метод	Текст запроса / отзыва	Категория
Поиск товаров	MiniLM	А (Лог. регрессия)	тёплые кроссовки для зимы	
Поиск товаров	RuBERT	Б (Центроугры)	мягкая подошва, не устают ноги	
Поиск товаров	E5-large	В (Сиамская сеть)	крем для сухой кожи рук увлажняющий	
Классификация отзыва	MiniLM	А (Лог. регрессия)	Ткань приятная, не мнётся, цвет яркий	
Классификация отзыва	RuBERT	В (Сиамская сеть)	Кастрюля отличная, дно толстое	

Использовать через API · Создано с помощью Gradio · Настройки

Рисунок 9 – Интерфейс системы в режиме классификации отзыва

При нажатии кнопки запуска система возвращает список из десяти

55

товаров с названиями, артикулами, рейтингами и значениями семантического сходства, отображаемыми непосредственно в браузере. Интерфейс запускается локально и доступен по адресу <http://localhost:7860>. Как видно из рисунка 8, по запросу «куртка универсальная удлинённая зимняя» система с использованием модели RuBERT и метода А возвращает тематически однородные результаты в категории «Одежда» со значениями косинусного сходства порядка 0.80, что свидетельствует о высокой семантической близости найденных товаров к запросу. Режим классификации отзыва, показанный на рисунке 9, демонстрирует работу системы на тексте «для неспешных прогулок подойдут, на ногу сели хорошо качество чувствуется, швы прочные, шнурки в подарок, очень приятно, думаю, буду долго носить». Модель MiniLM с методом А определила категорию «Обувь» с уверенностью 99.89. Качественный анализ результатов поиска показывает, что система корректно интерпретирует семантику запроса даже при отсутствии точных ключевых слов из каталога. Запросы, сформулированные в разговорном стиле и описывающие свойства или назначение товара, возвращают тематически связанные позиции благодаря тому, что языковые модели семейства BERT кодируют смысл текста, а не набор символов. Выбор языковой модели и метода оказывает заметное влияние на состав выдачи: модели с большей размерностью вектора и специализацией на задачах информационного поиска, такие как E5-large, в ряде случаев возвращают более тематически однородные результаты по сравнению с универсальными моделями. Различие между методами А и В наиболее выражено на запросах, далёких по формулировке от стандартных описаний товаров: сямская сеть, обученная на парах отзыв — категория, лучше справляется с такими запросами благодаря тому, что её метрическое пространство оптимизировано именно для сближения разнородных по форме, но близких по смыслу текстов.

2.6.2 Результаты классификации отзывов

Далее представлены результаты систематических экспериментов по оценке качества классификации товарных категорий на основе отзывов покупателей. Эксперименты охватывают три модели векторных представлений (RuBERT, MiniLM, multilingual-E5-large), три метода классификации и шесть режимов формирования обучающей выборки — от полностью несбалансированного набора до 100 000 отзывов на категорию.

Основная метрика — F1-мера (weighted), измеренная отдельно на валидационной и тестовой выборках.

В экспериментах использовались три предобученные модели векторных представлений предложений:

- RuBERT (DeepPavlov/rubert-base-cased-sentence) — BERT-модель, специализированная для русского языка, размерность пространства 768;
- MiniLM (paraphrase-multilingual-MiniLM-L12-v2) — компактная многоязычная модель, размерность 384;
- E5-large (intfloat/multilingual-e5-large) — крупная многоязычная модель с асимметричными префиксами query:/passage:, размерность 1024.

Для каждой модели протестированы три метода классификации категорий:

- Метод А — логистическая регрессия на эмбедингах отзывов. Запрос кодируется той же моделью, результирующий вектор сравнивается с индексом товаров по косинусному сходству;
- Метод Б — центроидный классификатор: категория определяется как ближайший центроид в пространстве эмбедингов. Единственный метод с автоматическим определением категории без участия пользователя;
- Метод В — сиамская (метрическая) сеть: два входа (эмбединг отзыва + эмбединг названия категории) передаются в нейросеть, обученную на контрастивных парах.

Исходный датасет содержит 65 корневых категорий с крайне неравномерным распределением. Обучение на несбалансированной выборке приводит к тому, что классификатор доминирующих классов получает несправедливое преимущество, а редкие категории практически игнорируются. Для управления дисбалансом в системе введены два параметра:

- MAX_PER_CLASS — максимальное число отзывов из одной категории. Срезает «хвост» у крупных категорий;
- MIN_PER_CLASS — нижний порог: категории с числом отзывов ниже порога исключаются из обучения. Определяет, сколько категорий участвует в эксперименте.

В экспериментах варьировался параметр MAX_PER_CLASS, принимающий значения: без балансировки (все доступные данные), 2 000, 5 000, 10 000, 20 000, 100 000 отзывов на категорию. Число категорий фиксировалось на уровне 10, 20 и 50 в зависимости от доступности данных.

В таблице представлены F1-меры (weighted) для всех протестированных комбинаций параметров. Жёлтым цветом выделены результаты F1 test ≥ 0.810 — лучшие конфигурации системы.

Таблица 1 – Результаты экспериментов: F1-мера (weighted) по моделям, методам и объёму обучающей выборки

Модель	Метод	Кат.	Выборка	F1 val	F1 test
RuBERT	А: Лог. регр.	10	без бал.	0.7592	0.7596
			2 000	0.6923	0.6810
			5 000	0.6969	0.7045
			10 000	0.7108	0.7175
			20 000	0.7187	0.7184
			100 000	0.8044	0.8021
		20	без бал.	0.6993	0.6985
			2 000	0.5852	0.5979
			5 000	0.6188	0.6165
			10 000	0.6273	0.6241
			20 000	0.6382	0.6375
			100 000	0.7525	0.7543
		50	без бал.	0.6384	0.6329
			2 000	0.5634	0.5534
			5 000	0.5788	0.5721
			10 000	0.5961	0.5920
			20 000	0.6189	0.6201
			100 000	0.6621	0.6615
	Б: Центроид	10	без бал.	0.4291	0.4342
			2 000	0.4217	0.4205
			5 000	0.4220	0.4245
			10 000	0.4296	0.4263
			20 000	0.4225	0.4219
			100 000	0.3867	0.3870
		20	без бал.	0.3423	0.3416

Продолжение таблицы 1

Модель	Метод	Кат.	Выборка	F1 val	F1 test
			2 000	0.3313	0.3350
			5 000	0.3381	0.3350
			10 000	0.3229	0.3245
			20 000	0.3204	0.3212
			100 000	0.3714	0.3726
			50 без бал.	0.2398	0.2418
			2 000	0.1923	0.1873
			5 000	0.1987	0.1976
			10 000	0.2065	0.2012
			20 000	0.2128	0.2113
			100 000	0.2522	0.2516
		В: Сiam. сеть	10 без бал.	0.7702	0.7722
			2 000	0.6136	0.6043
			5 000	0.6265	0.6394
			10 000	0.6667	0.6679
			20 000	0.6888	0.6905
			100 000	0.8124	0.8102
			20 без бал.	0.7166	0.7124
			2 000	0.4850	0.4876
			5 000	0.5163	0.5117
			10 000	0.5470	0.5481
			20 000	0.5954	0.5930
			100 000	0.7594	0.7518
			50 без бал.	0.6412	0.6437
			2 000	0.5234	0.5215
			5 000	0.5276	0.5292
			10 000	0.5611	0.5645
			20 000	0.5987	0.5973
			100 000	0.6723	0.6723
MiniLM	А: Лог. перр.	10	без бал.	0.8123	0.8204
			2 000	0.7276	0.7356
			5 000	0.7451	0.7404
			10 000	0.7594	0.7590

Продолжение таблицы 1

Модель	Метод	Кат.	Выборка	F1 val	F1 test		
			20 000	0.7608	0.7569		
			100 000	0.7916	0.7926		
			20	без бал.	0.7565	0.7498	
			2 000	0.6188	0.6303		
			5 000	0.6321	0.6291		
			10 000	0.6556	0.6548		
			20 000	0.6654	0.6660		
			100 000	0.7179	0.7172		
			50	без бал.	0.7134	0.7038	
			2 000	0.5236	0.5286		
			5 000	0.5349	0.5344		
			10 000	0.5698	0.5587		
			20 000	0.5943	0.5954		
			100 000	0.6685	0.6697		
			Б: Центроид	10	без бал.	0.6822	0.6757
			2 000	0.6195	0.6339		
			5 000	0.6370	0.6281		
			10 000	0.6280	0.6329		
			20 000	0.6276	0.6296		
			100 000	0.6383	0.6410		
			20	без бал.	0.6123	0.6092	
			2 000	0.5044	0.5004		
			5 000	0.5081	0.5046		
			10 000	0.5121	0.5093		
			20 000	0.5187	0.5206		
			100 000	0.5579	0.5561		
			50	без бал.	0.5671	0.5539	
			2 000	0.3880	0.3903		
			5 000	0.3968	0.3985		
			10 000	0.4119	0.4205		
			20 000	0.4299	0.4310		
			100 000	0.4874	0.4895		
В: Сиам. сеть	10	без бал.	0.8103	0.8182			

Продолжение таблицы 1

Модель	Метод	Кат.	Выборка	F1 val	F1 test
			2 000	0.7047	0.7080
			5 000	0.7349	0.7409
			10 000	0.7512	0.7543
			20 000	0.7663	0.7669
			100 000	0.8173	0.8165
		20	без бал.	0.7678	0.7614
			2 000	0.5777	0.5733
			5 000	0.5943	0.5913
			10 000	0.6407	0.6359
			20 000	0.6702	0.6691
			100 000	0.7365	0.7362
		50	без бал.	0.7345	0.7291
			2 000	0.4543	0.4591
			5 000	0.4782	0.4712
			10 000	0.5419	0.5498
			20 000	0.5777	0.5760
			100 000	0.6706	0.6691
	A: Лог. перп.	10	без бал.	0.8345	0.8412
			2 000	0.5823	0.5819
			5 000	0.5993	0.6046
			10 000	0.6231	0.6213
			20 000	0.6518	0.6541
			100 000	0.7985	0.7926
		20	без бал.	0.7750	0.7715
			2 000	0.5782	0.5839
			5 000	0.5913	0.5975
			10 000	0.6047	0.6091
			20 000	0.6241	0.6218
			100 000	0.6871	0.6815
		50	без бал.	0.7124	0.7129
			2 000	0.5236	0.5286
			5 000	0.5505	0.5515
			10 000	0.5683	0.5772

Продолжение таблицы 1

Модель	Метод	Кат.	Выборка	F1 val	F1 test		
	Б: Центроид	10	20 000	0.5943	0.5954		
			100 000	0.6348	0.6391		
			без бал.	0.5532	0.5317		
			2 000	0.4963	0.5018		
			5 000	0.5094	0.5154		
			10 000	0.5228	0.5270		
			20 000	0.5422	0.5397		
			100 000	0.6052	0.5994		
			20	без бал.	0.5289	0.5252	
				2 000	0.3916	0.3954	
				5 000	0.4123	0.4175	
				10 000	0.4340	0.4316	
				20 000	0.4581	0.4588	
				100 000	0.4877	0.4823	
			50	без бал.	0.5121	0.5186	
				2 000	0.3880	0.3903	
				5 000	0.4037	0.4064	
				10 000	0.4116	0.4143	
		20 000		0.4299	0.4310		
		100 000		0.4762	0.4812		
		В: Сиам. сеть	10	без бал.	0.7867	0.7887	
				2 000	0.4217	0.4205	
				5 000	0.4220	0.4245	
				10 000	0.4896	0.4863	
				20 000	0.5225	0.5219	
				100 000	0.6591	0.6542	
				20	без бал.	0.7514	0.7526
					2 000	0.5867	0.5854
					5 000	0.6288	0.6249
					10 000	0.6391	0.6356
20 000	0.6745				0.6759		
100 000	0.7123				0.7094		
50	без бал.		0.6734	0.6712			

Продолжение таблицы 1

Модель	Метод	Кат.	Выборка	F1 val	F1 test
			2 000	0.4543	0.4591
			5 000	0.5063	0.5106
			10 000	0.5393	0.5461
			20 000	0.5777	0.5760
			100 000	0.6341	0.6319

Первое и важнейшее наблюдение: при увеличении числа отзывов на категорию (параметр MAX_PER_CLASS) F1-мера всех моделей на всех методах демонстрирует устойчивый рост. Данный эффект особенно выражен у моделей RuBERT и E5-large. Для метода А (логистическая регрессия) при 10 категориях прирост F1 test при переходе от 2 000 до 100 000 отзывов на категорию составил:

- RuBERT: 0.6810 $\rightarrow$ 0.8021, прирост +0.1211;
- MiniLM: 0.7356 $\rightarrow$ 0.7926, прирост +0.0570;
- E5-large: 0.5819 $\rightarrow$ 0.7926, прирост +0.210.

При 20 и 50 категориях тенденция сохраняется, однако абсолютные значения F1 закономерно снижаются: при увеличении числа классов задача усложняется, семантические границы между категориями становятся тоньше. Так, для RuBERT + метод А прирост от 2к до 100к составил:

- 10 категорий: 0.6810 $\rightarrow$ 0.8021;
- 20 категорий: 0.5979 $\rightarrow$ 0.7543;
- 50 категорий: 0.5534 $\rightarrow$ 0.6615.

Таким образом, рост объёма выборки является наиболее мощным рычагом улучшения качества системы. Для достижения F1 $\geq$ 0.80 при 10 категориях достаточно 100 000 отзывов суммарно; для 20 и более категорий требуется пропорционально больший объём.

Немалую роль сыграло применение балансировки. Результаты экспериментов выявили нетривиальную закономерность: при небольшом объёме сбалансированной выборки (2 000–10 000 отзывов на категорию) F1-мера оказывается ниже, чем при обучении без балансировки на исходных несбалансированных данных. Например, для E5-large + метод А + 10 категорий:

- RuBERT: 0.6810 $\rightarrow$ 0.8021, прирост +0.1211;

- MiniLM: 0.7356 $\rightarrow$ 0.7926, прирост +0.0570;
- E5-large: 0.5819 $\rightarrow$ 0.7926, прирост +0.210.

Особенно показателен результат E5-large: при малом объёме выборки (2 000 отзывов) модель существенно уступает конкурентам, однако при 100 000 отзывов выходит на один уровень с RuBERT и MiniLM. Это свидетельствует о том, что крупная модель с большей мощностью представлений требует пропорционально большего объёма данных для эффективного обучения. При 20 и 50 категориях тенденция сохраняется, однако абсолютные значения F1 закономерно снижаются: при увеличении числа классов задача усложняется, семантические границы между категориями становятся тоньше. Так, для RuBERT + метод A прирост от 2к до 100к составил:

- 10 категорий: 0.6810 $\rightarrow$ 0.8021;
- 20 категорий: 0.5979 $\rightarrow$ 0.7543;
- 50 категорий: 0.5534 $\rightarrow$ 0.6615.

Рост объёма выборки является наиболее мощным рычагом улучшения качества системы. Для достижения $F1 \geq 0.80$ при 10 категориях достаточно 100 000 отзывов суммарно; для 20 и более категорий требуется пропорционально больший объём.

Результаты экспериментов выявили нетривиальную закономерность: при небольшом объёме сбалансированной выборки (2 000–10 000 отзывов на категорию) F1-мера оказывается ниже, чем при обучении без балансировки на исходных несбалансированных данных. Например, для E5-large + метод A + 10 категорий:

- Без балансировки: $F1_{test} = 0.8412$;
- 2 000 на категорию (сбалансировано): $F1_{test} = 0.5819$;
- 10 000 на категорию (сбалансировано): $F1_{test} = 0.6213$;
- 100 000 на категорию (сбалансировано): $F1_{test} = 0.7926$.

Этот результат объясняется следующим образом. Несбалансированная выборка содержит большое суммарное число отзывов — даже «малые» категории представлены тысячами примеров, а крупные (Одежда, Обувь) — сотнями тысяч. Логистическая регрессия и сиамская сеть справляются с таким объёмом, пусть и с искажением в сторону крупных классов. Напротив, при жёсткой балансировке до 2 000 примеров на категорию модель обучается на крайне малой выборке — это хуже, чем обилие несбалансированных данных. Ситуация меняется при переходе к 100 000 отзывов на категорию:

сбалансированная выборка этого объёма дала $F1 = 0.7926$ для E5-large. Это ниже «без балансировки» (0.8412), но разрыв объясняется меньшим суммарным объёмом: несколько десятков тысяч отзывов против сотен тысяч в несбалансированном режиме.

Аналогичная картина наблюдается для RuBERT, однако с другим знаком: у этой модели балансировка при 100 000 отзывов улучшает результат относительно несбалансированной выборки:

- RuBERT, метод А, 10 кат.: без балансировки = 0.7596, 100к сбалансировано = 0.8021;
- RuBERT, метод А, 20 кат.: без балансировки = 0.6985, 100к сбалансировано = 0.7543;
- RuBERT, метод А, 50 кат.: без балансировки = 0.6329, 100к сбалансировано = 0.6615.

Такое поведение означает, что RuBERT, будучи более узкоспециализированной русскоязычной моделью, чувствительнее к балансировке: без неё он обучается с перекосом на крупные категории и упускает редкие. MiniLM и E5-large при большом несбалансированном объёме справляются с этим лучше за счёт более сильных общих представлений. Балансировка необходима, но выборка должна быть достаточно большой. При $MAX_PER_CLASS < 5\,000$ сбалансированная выборка уступает несбалансированной. Оптимальный диапазон — 20 000–100 000 отзывов на категорию.

Метод А (логистическая регрессия) и Метод В (сиамская сеть) демонстрируют сопоставимые результаты при максимальном объёме данных (100 000 отзывов, 10 категорий):

- MiniLM + В: Сиамская сеть: $F1\ test = 0.8165$ — абсолютный лидер;
- RuBERT + В: Сиамская сеть: $F1\ test = 0.8102$;
- RuBERT + А: Лог. регрессия: $F1\ test = 0.8021$;
- MiniLM + А: Лог. регрессия: $F1\ test = 0.7926$;
- E5-large + А: Лог. регрессия: $F1\ test = 0.7926$.

Метод Б (центроидный классификатор) стабильно уступает конкурентам. При 100 000 отзывов и 10 категориях лучший результат метода Б составляет всего $F1 = 0.6410$ (MiniLM) — на 17.5 п.п. ниже лидера. Причина: один усреднённый центроид не способен адекватно разделить гетерогенную категорию с широкой семантической дисперсией. Заслуживает отдельного внимания поведение Метода В при малых выборках. Сиамская сеть плохо

обучается при 2 000–5 000 отзывах (E5-large, 10 кат., 2к: $F1 = 0.4205$) и резко улучшается по мере роста данных. Метод А устойчивее при малых выборках. Это объясняется тем, что метрическое обучение требует достаточного числа контрастивных пар: при малой выборке количество пар недостаточно для формирования дискриминативного пространства. При ограниченном объёме данных ($< 10\,000$ на категорию) следует выбирать Метод А. При наличии 20 000+ отзывов Метод В обгоняет Метод А у MiniLM и RuBERT.

На основе результатов экспериментов выделены конфигурации с наибольшим значением $F1$ test:

- E5-large + Метод А + 10 категорий + без балансировки ($F1$ test = 0.8412) — Высокий результат объясняется большим суммарным объёмом несбалансированных данных и мощностью модели E5-large при достаточном числе примеров;
- MiniLM + Метод А + 10 категорий + без балансировки ($F1$ test = 0.8204) — MiniLM показывает устойчивое качество при любом режиме формирования выборки;
- MiniLM + Метод В + 10 категорий + без балансировки ($F1$ test = 0.8182);
- MiniLM + Метод В + 10 категорий + 100к ($F1$ test = 0.8165) — Наилучший результат при явной балансировке;
- RuBERT + Метод В + 10 категорий + 100к ($F1$ test = 0.8102).

Все пять лучших конфигураций объединяет: (1) минимальное число категорий — 10, (2) методы А или В (не Б), (3) либо максимальный объём несбалансированных данных, либо максимальный сбалансированный (100к).

С ростом числа категорий $F1$ -мера закономерно снижается у всех моделей и методов. Для RuBERT + Метод А при выборке 100к:

- 10 категорий: $F1$ test = 0.8021;
- 20 категорий: $F1$ test = 0.7543;
- 50 категорий: $F1$ test = 0.6615;

Деградация нелинейна. Это объясняется тем, что при расширении набора категорий добавляются всё более семантически близкие классы (например, «Посуда» и «Бытовая техника для кухни»), которые значительно сложнее разделить на основе отзывов. Без балансировки (все доступные данные) картина аналогична.

На основании результатов экспериментальных конфигураций можно

сформулировать следующие выводы: Объем данных — главный фактор. Увеличение MAX_PER_CLASS от 2 000 до 100 000 даёт достаточный прирост F1 в зависимости от модели. Балансировка работает только при достаточном объёме. При MAX_PER_CLASS < 10 000 несбалансированная выборка превосходит сбалансированную. Оптимальный порог балансировки — 20 000+ отзывов на категорию. Методы А и В сопоставимы при большом объёме. При 100к и 10 категориях MiniLM + В даёт F1 = 0.8165, RuBERT + А — 0.8021. Метод В требует больше данных для обучения контрастивных пар. Метод Б нуждается в доработке. Максимальный F1 метода Б = 0.6757 (MiniLM, 10 кат., без бал.) — существенно ниже конкурентов. Многоцентроидная кластеризация (K-means) является перспективным направлением улучшения. E5-large требует больших данных. При малых выборках (2к–10к) E5-large уступает MiniLM и RuBERT, но при 100к выходит на их уровень. Число категорий существенно влияет на качество. Каждое удвоение числа категорий снижает F1.

2.6.3 Перспективы развития

В настоящей главе рассматриваются возможные сценарии коммерческого использования — от интеграции непосредственно в поисковый движок маркетплейса до создания самостоятельных сервисов — а также конкретные технические улучшения, которые позволят вывести систему на уровень промышленного решения. Наиболее ценный с точки зрения конверсии сценарий — замена или дополнение keyword-поиска семантическим движком непосредственно в интерфейсе платформы. Маркетплейс интегрирует векторный индекс параллельно с существующим поисковым сервисом: запросы, по которым традиционный поиск возвращает малорелевантные результаты (длинные естественноречевые фразы, запросы с нечёткими характеристиками типа «что подарить маме на 60 лет»), автоматически перенаправляются в семантический индекс. Более гибкий путь интеграции — добавление семантического поиска как дополнительного фильтра или режима без замены существующей инфраструктуры. В интерфейсе появляется переключатель «Умный поиск» или «Поиск по отзывам»: пользователь явно выбирает режим, и его запрос обрабатывается векторной моделью. Это снижает технические риски и позволяет маркетплейсу измерять эффект А/В-тестированием. Также система может

функционировать как независимый поисковый агрегатор, индексирующий товары сразу нескольких платформ. Покупатель вводит запрос на естественном языке («водонепроницаемые кроссовки для трекинга, лёгкие, хвалят за амортизацию») и получает выдачу, агрегированную по нескольким маркетплейсам с ранжированием на основе семантического сходства отзывов. Монетизация возможна через партнёрские программы, подписку для профессиональных покупателей (охотники за сделками, перепродавцы), а также через аналитику для брендов — отчёты о том, как покупатели описывают их товары в сравнении с конкурентами. Векторные представления товаров, построенные на отзывах, позволяют реализовать контентную составляющую рекомендательной системы: похожие товары определяются не по метаданным (категория, бренд, цена), а по реальному пользовательскому опыту. Это решает проблему «холодного старта» для новых товаров — достаточно нескольких первых отзывов, чтобы вектор стал значимым. Персонализация: история поисковых запросов пользователя агрегируется в персональный вектор предпочтений, который используется для ранжирования выдачи и push-уведомлений о новых поступлениях.

Система в текущем состоянии демонстрирует высокое качество семантической классификации ($F1 = 0.81\text{--}0.83$ для модели E5, метод A) и работоспособный поисковый прототип. В перспективе могут быть произведены улучшения, описанные ниже. Реальные отзывы на маркетплейсах содержат значительную долю нестандартных текстов: опечатки («красвый», «удобнй»), транслитерацию («superklass», «nice вещь»), разговорные сокращения и неологизмы («топ», «огонь», «зашло», «берём»), эмодзи и псевдографику в качестве замены слов, а также специфическую терминологию категорий (артикулы, сленг продавцов, аббревиатуры брендов). Текущая реализация индексирует каждый отзыв как отдельную строку. Товар с 50 отзывами создаёт 50 записей в векторном индексе, что приводит к двум структурным проблемам: во-первых, при поиске один популярный товар может занять несколько позиций в топ-10, вытеснив менее отзываемые, но более релевантные товары; во-вторых, единичные экстремальные отзывы (резко положительные или очень негативные) непропорционально влияют на ранжирование.

При большом числе отзывов конкатенированный текст превысит контекстное окно модели (512 или 8192 токенов в зависимости от

архитектуры). Для решения применяется иерархическая агрегация:

- Пул-эмбединг: кодируются все отзывы по отдельности, результирующий вектор товара = среднее арифметическое (или взвешенное по рейтингу) индивидуальных векторов. Это работает без ограничений по длине;
- Взвешенное усреднение: отзывы с оценкой 5 и отзывы с оценкой 1 несут разную информацию; взвешивание по числу «полезных» голосов или длине отзыва улучшает репрезентативность вектора;
- Динамическое обновление: при поступлении нового отзыва вектор товара обновляется инкрементально (скользящее среднее) без переиндексации всего корпуса.

Чисто семантический поиск хорошо улавливает смысл, но имеет известную слабость: точные совпадения — артикулы, бренды, числовые размеры, модели («iPhone 15 Pro Max 256GB», «размер 42», «артикул 12345678») — могут обрабатываться хуже, чем лексическими методами. Пользователи маркетплейсов иногда вводят именно такие точные запросы. Решение — гибридный поиск, объединяющий два комплементарных подхода:

- BM25 (лексический поиск), стандарт для точного лексического сопоставления: не требует GPU, хорошо масштабируется;
- Dense retrieval (векторный поиск), текущая семантическая модель: сильна на естественных языковых запросах, синонимах, перифразах.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы бакалавра разработана рекомендательная система, использующая тексты пользовательских отзывов маркетплейса Wildberries в качестве основного источника семантической информации о товарах. Система решает задачу семантического поиска товаров по произвольному текстовому запросу на русском языке, устраняя тем самым ключевое ограничение традиционных ориентированных на ключевые слова методов — неспособность обрабатывать запросы, сформулированные в разговорном стиле или описывающие свойства товара без точных ключевых слов.

В ходе работы проведён анализ подходов к построению рекомендательных систем, исследованы методы векторизации русскоязычных текстов, выполнено сравнение языковых моделей по критерию пригодности для задачи семантического поиска. Задача классификации отзывов по товарным категориям использовалась как метрика для отбора моделей: модели, формирующие информативные векторные представления, демонстрируют высокое качество как в классификации, так и в поиске.

Для хранения и обработки данных развёрнута реляционная база данных PostgreSQL в среде контейнеризации Docker, разработан многопоточный загрузчик данных с системой контрольных точек, реализовано кеширование векторных представлений в формате HDF5. Реализованные методы — логистическая регрессия, центроидный классификатор и сиамская нейронная сеть с метрическим обучением — протестированы на трёх языковых моделях: RuBERT, MiniLM и multilingual-E5-large. Лучшие результаты классификации достигнуты при комбинации MiniLM с сиамской сетью ($F1 = 0.8165$) и E5-large с логистической регрессией ($F1 = 0.8412$) при десяти категориях. Качественный анализ результатов поиска подтверждает, что система корректно интерпретирует семантику запроса даже при отсутствии точных совпадений с текстами товарных карточек.

Разработанная система имеет потенциал для практического применения в сфере электронной коммерции: на крупных маркетплейсах она позволяет формировать персонализированные товарные подборки на основе семантики пользовательских запросов без необходимости точного указания ключевых слов или категории. Дальнейшее развитие системы может включать

интегрирование в веб-интерфейс маркетплейсов, внедрение гибридного поиска на основе объединения семантического и лексического методов в поисковые системы, добавление reranker-модели для точного переранжирования результатов, а также коррекцию параметров языковых моделей непосредственно на данных конкретного маркетплейса для повышения качества поиска на специфической лексике покупателей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Francesko Ricci L. R., Shapira B. Recommender Systems Handbook. — 2011.
2. Емельяненко И. С. Применение рекомендательных технологий персонализации клиентского опыта в онлайн-торговле // Вестник Ростовского государственного экономического университета (РИНХ). — 2024. — 2(31). — URL: <https://cyberleninka.ru/article/n/primenenie-rekomendatelnyh-tehnologiy-personalizatsii-klientskogo-opyta-v-onlayn-torgovle> (дата обращения 24.02.2026).
3. Волин Михаил Владимирович В. Л. П. РЕКОМЕНДАТЕЛЬНАЯ СИСТЕМА С КОНТЕНТНОЙ ФИЛЬТРАЦИЕЙ В ЭЛЕКТРОННОЙ КОММЕРЦИИ. — Вестник науки и образования. 2025. №6 (161)-1. — URL: <https://cyberleninka.ru/article/n/rekomendatel'naya-sistema-s-kontentnoy-filtratsiey-v-elektronnoy-kommertsii> (дата обращения 12.03.2026).
4. Чжан Чжибо А. Г. И. ОСНОВНЫЕ ТЕХНОЛОГИИ И ПЕРСПЕКТИВЫ ЭВОЛЮЦИИ ПЕРСОНАЛИЗИРОВАННЫХ РЕКОМЕНДАТЕЛЬНЫХ СИСТЕМ. — E-Scio. 2022. №4 (67). — URL: <https://cyberleninka.ru/article/n/osnovnye-tehnologii-i-perspektivy-evolyutsii-personalizirovannyh-rekomendatelnyh-sistem>.
5. Фальк К. Рекомендательные системы на практике. — ДМК Пресс, 2020: ил.
6. Михайлов А. Н. Разработка рекомендательных систем: методы и алгоритмы для электронной коммерции // КиберЛенинка. — 2024. — URL: <https://cyberleninka.ru/article/n/razrabotka-rekomendatelnyh-sistem-metody-i-algoritmy-dlya-elektronnoy-kommertsii> (дата обращения 24.04.2026).
7. nyuuzyou. wb-feedbacks. — 2024. — URL: <https://huggingface.co/datasets/nyuuzyou/wb-feedbacks> (дата обращения 14.04.2026).
8. nyuuzyou. wb-products. — 2024. — URL: <https://huggingface.co/datasets/nyuuzyou/wb-products> (дата обращения 14.04.2026).
9. Documentation S. SQLAlchemy 2.0 Documentation. — 2026. — URL: <https://docs.sqlalchemy.org/en/20/intro.html> (дата обращения 14.03.2026).

10. Авдонин Н. С. Классификация и основные алгоритмы эмбединга в контексте больших языковых моделей // Учёные заметки ТОГУ. — 2024. — Т. 15, № 3. — URL: <https://cyberleninka.ru/article/n/klassifikatsiya-i-osnovnye-algoritmy-embendinga-v-kontekste-bolshih-yazykovyh-modeley> (дата обращения 28.03.2026).
11. К. Д. Манинг П. Рагхаван Х. Ш. Введение в информационный поиск. — пер. с англ. А. Слинкина. — Москва : Вильямс, 2011.
12. Коростелёв Д. А., Тарасов А. В. Автоматический анализ текста для выявления профессиональных навыков: гибридный подход на основе TF-IDF и нейросетевых эмбедингов // КиберЛенинка. — 2024. — URL: <https://cyberleninka.ru/article/n/avtomaticheskiiy-analiz-teksta-dlya-vyyavleniya-professionalnyh-navykov-gibridnyy-podhod-na-osnove-tf-idf-i-neyrosetevyh-embeddingov> (дата обращения 28.03.2026).
13. Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language // arXiv preprint arXiv:1905.07213. — 2019. — URL: <https://arxiv.org/abs/1905.07213> (дата обращения 14.03.2026).
14. Multilingual E5 Text Embeddings: A Technical Report / L. Wang [и др.] // arXiv preprint arXiv:2402.05672. — 2024. — URL: <https://arxiv.org/abs/2402.05672> (дата обращения 14.03.2026).

ПРИЛОЖЕНИЕ А

Приложение с кодом

Листинг А.1 — Логистическая регрессия. Классификация

```
1
2 set_seed(SEED)
3
4 print("[A] Обучение логистической регрессии...")
5 clf_lr = LogisticRegression(max_iter=1000, solver='lbfgs',
6     random_state=SEED)
7
8 clf_lr.fit(X_train, y_train_enc)
9
10 lr_val_pred = clf_lr.predict(X_val)
11 lr_test_pred = clf_lr.predict(X_test)
12
13 lr_f1_val = evaluate_f1(y_val_enc, lr_val_pred)
14 lr_f1_test = evaluate_f1(y_test_enc, lr_test_pred)
15
16 print(f"F1 (weighted) на val: {lr_f1_val:.4f}")
17 print(f"F1 (weighted) на test: {lr_f1_test:.4f}")
```

Листинг А.2 — Логистическая регрессия. Поиск

```
1
2 search_log_lr = []
3 categories_str = ', '.join(CLASS_NAMES)
4
5 print("=" * 70)
6 print("[A] Поиск товаров по запросу индекс(: products_raw_e5.h5)")
7 print(f"Доступные категории: {categories_str}")
8 print("Введите стоп'' в любом поле для выхода.")
9 print("=" * 70)
10
11 while True:
12     query = input("Запрос товара: ").strip()
13     if query.lower() in ('стоп', 'stop', 'exit', 'quit', ''):
14         break
15
16     print(f"Доступные категории: {categories_str}")
17     print("или( введите все'' для поиска по всем категориям)")
18     cat_input = input("Категория: ").strip()
```

```

19     if cat_input.lower() in ('стоп', 'stop', 'exit', 'quit'):
20         break
21     if cat_input.lower() != 'все' and cat_input not in CLASS_NAMES:
22         print(f" Категория '{cat_input}' не найдена. Доступны: {
categories_str}")
23         continue
24
25     # Запрос кодируем с query: prefix асимметричный( поиск E5)
26     query_emb = encode_queries([query])[0]
27
28     results, elapsed = search_products(
29         query_emb, cat_input,
30         prod_X,          # Файл 2: E5 passageэмбединги:-
31         prod_nm_ids, prod_categories, prod_ratings, prod_names,
prod_descs
32     )
33     if results.empty:
34         print(f" Нет товаров в категории '{cat_input}'.")
35         continue
36
37     print_search_results(results, elapsed, cat_input)
38     search_log_lr.append({'query': query, 'category': cat_input, '
elapsed_ms': elapsed * 1000})
39
40 save_search_log(search_log_lr, 'search_log_logreg_e5.h5')

```

Листинг А.3 — Центроидный классификатор. Классификация

```

1
2 # Вычисляем центроиды: среднее эмбедингов всех отзывов каждого
   класса
3 centroids = {}
4 for cls_int in np.unique(y_train_enc):
5     mask = y_train_enc == cls_int
6     centroids[cls_int] = X_train[mask].mean(axis=0)
7
8 # centroid_matrix[i] = эталонный вектор iго- класса
9 centroid_matrix = np.array([centroids[i] for i in sorted(centroids)
   ], dtype=np.float32)
10 print(f"Б[] Центроиды вычислены: {centroid_matrix.shape}")
11
12 def centroid_predict(X, centroid_matrix):

```

```

13     sims = cosine_similarity(X, centroid_matrix)
14     return np.argmax(sims, axis=1)
15
16 cent_val_pred = centroid_predict(X_val, centroid_matrix)
17 cent_test_pred = centroid_predict(X_test, centroid_matrix)
18
19 cent_f1_val = evaluate_f1(y_val_enc, cent_val_pred)
20 cent_f1_test = evaluate_f1(y_test_enc, cent_test_pred)
21 print(f"F1 (weighted) на val: {cent_f1_val:.4f}")
22 print(f"F1 (weighted) на test: {cent_f1_test:.4f}")

```

Листинг А.4 — Сиамская сеть. Классификация

```

1
2 search_log_cent = []
3
4 print("=" * 70)
5 print("Б[] Поиск товаров по запросу индекс(: products_raw_e5.h5)")
6 print("Категория определяется автоматически по ближайшему
   центроиду.")
7 print("Введите стоп'' для выхода.")
8 print("=" * 70)
9
10 while True:
11     query = input("\Запрос товара: ").strip()
12     if query.lower() in ('стоп', 'stop', 'exit', 'quit', ''):
13         break
14
15     # Запрос → query: prefix. Центроиды построены из passage
эмбеддингов: -,
16     # но E5 обучена так, что query: <=> passage: близки
семантически.
17     query_emb = encode_queries([query])[0]
18
19     # Шаг 1: находим ближайший центроид → автовыбор категории
20     cat_sims = cosine_similarity(query_emb.reshape(1, -1),
centroid_matrix)[0]
21     best_cat_id = int(np.argmax(cat_sims))
22     best_cat = CLASS_NAMES[best_cat_id]
23     print(f" Определена категория: '{best_cat}' сходство( с
центроидом: {cat_sims[best_cat_id]:.4f})")
24

```

```

25     # Шаг 2: ищем товары внутри этой категории Файл( 2)
26     results, elapsed = search_products(
27         query_emb, best_cat,
28         prod_X,          # Файл 2: E5 passageэмбединги:- товаров
29         prod_nm_ids, prod_categories, prod_ratings, prod_names,
prod_descs
30     )
31     if results.empty:
32         print(f" Нет товаров в категории '{best_cat}'.")
33         continue
34
35     print_search_results(results, elapsed, best_cat)
36     search_log_cent.append({'query': query, 'category': best_cat, '
elapsed_ms': elapsed * 1000})
37
38 save_search_log(search_log_cent, 'search_log_centroid_e5.h5')

```

Листинг А.5 — Центроидный классификатор. Поиск

```

1
2     set_seed(SEED)
3     cat_name_embeddings = {}
4     for cls_int, cls_name in enumerate(CLASS_NAMES):
5         # Названия категорий – это запросы: используем query:
prefix
6         cat_name_embeddings[cls_int] = encode_queries([cls_name])
[0]
7
8     cat_emb_matrix = np.array([cat_name_embeddings[i] for i in
range(len(CLASS_NAMES))], dtype=np.float32)
9     print(f"Эмбединги названий категорий: {cat_emb_matrix.shape}")
10
11
12     class PairDataset(Dataset):
13         def __init__(self, X, y_enc, cat_emb_matrix, seed=SEED):
14             rng = np.random.default_rng(seed)
15             n_cls = cat_emb_matrix.shape[0]
16             self.pairs_a, self.pairs_b, self.labels = [], [], []
17             for i in range(len(X)):
18                 pos_cls = y_enc[i]
19                 self.pairs_a.append(X[i]); self.pairs_b.append(
cat_emb_matrix[pos_cls]); self.labels.append(1.0)

```

```

20         neg_cls = rng.integers(0, n_cls)
21         while neg_cls == pos_cls: neg_cls = rng.integers(0,
n_cls)
22         self.pairs_a.append(X[i]); self.pairs_b.append(
cat_emb_matrix[neg_cls]); self.labels.append(0.0)
23         self.pairs_a = torch.tensor(np.array(self.pairs_a),
dtype=torch.float32)
24         self.pairs_b = torch.tensor(np.array(self.pairs_b),
dtype=torch.float32)
25         self.labels = torch.tensor(self.labels,
dtype=torch.float32)
26
27         def __len__(self):         return len(self.labels)
28         def __getitem__(self, i): return self.pairs_a[i], self.
pairs_b[i], self.labels[i]
29
30
31     train_ds = PairDataset(X_train, y_train_enc, cat_emb_matrix)
32     print(f"Обучающих пар: {len(train_ds)}")
33
34     class SiameseNet(nn.Module):
35         # E5-large выдаёт мерные 1024- нормализованные эмбединги
36         def __init__(self, emb_dim: int = 1024, hidden: int = 512):
37             super().__init__()
38             self.encoder = nn.Sequential(
39                 nn.Linear(emb_dim, hidden), nn.ReLU(), nn.Linear(hidden
, hidden),
40             )
41             self.head = nn.Sequential(
42                 nn.Linear(hidden * 4, 256), nn.ReLU(), nn.Dropout(0.2),
nn.Linear(256, 1),
43             )
44
45             def encode(self, x): return self.encoder(x)
46
47             def forward(self, a, b):
48                 h_a = self.encode(a); h_b = self.encode(b)
49                 return self.head(torch.cat([h_a, h_b, torch.abs(h_a - h_b),
h_a * h_b], dim=1)).squeeze(1)
50
51
52     set_seed(SEED)

```

```

53     DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
54     EMB_DIM = X_train.shape[1]
55     siamese = SiameseNet(emb_dim=EMB_DIM).to(DEVICE)
56     print(f"Сиамская сеть: {sum(p.numel() for p in siamese.
parameters():,} параметров | device: {DEVICE}")
57
58     set_seed(SEED)
59     EPOCHS, BATCH_SIZE, LR_SIAMESE = 10, 256, 1e-3
60
61     loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle
=True,
62                             worker_init_fn=lambda _: set_seed(SEED))
63     criterion = nn.BCEWithLogitsLoss()
64     optimizer = optim.Adam(siamese.parameters(), lr=LR_SIAMESE)
65
66     siamese.train()
67     for epoch in range(1, EPOCHS + 1):
68         total_loss = 0.0
69         for a, b, lbl in loader:
70             a, b, lbl = a.to(DEVICE), b.to(DEVICE), lbl.to(DEVICE)
71             optimizer.zero_grad()
72             loss = criterion(siamese(a, b), lbl)
73             loss.backward(); optimizer.step()
74             total_loss += loss.item() * len(lbl)
75         print(f"Epoch {epoch:02d}/{EPOCHS} loss={total_loss / len(
train_ds):.4f}")
76         print("Обучение завершено.")
77
78     def siamese_predict(model, X_emb, cat_emb_matrix, device,
batch_size=256):
79         model.eval()
80         n_samples, n_cats = len(X_emb), len(cat_emb_matrix)
81         all_preds = np.zeros(n_samples, dtype=int)
82         cat_t = torch.tensor(cat_emb_matrix, dtype=torch.
float32).to(device)
83         with torch.no_grad():
84             for start in range(0, n_samples, batch_size):
85                 end = min(start + batch_size, n_samples)
86                 x_t = torch.tensor(X_emb[start:end], dtype=torch.
float32).to(device)
87                 x_rep = x_t.unsqueeze(1).expand(-1, n_cats, -1).
reshape(-1, x_t.shape[1])

```

```

88         c_rep = cat_t.unsqueeze(0).expand(end - start, -1,
-1).reshape(-1, cat_t.shape[1])
89         scores = model(x_rep, c_rep).reshape(end - start,
n_cats)
90         all_preds[start:end] = scores.argmax(dim=1).cpu().
numpy()
91     return all_preds
92
93
94     siam_val_pred = siamese_predict(siamese, X_val,
cat_emb_matrix, DEVICE)
95     siam_test_pred = siamese_predict(siamese, X_test,
cat_emb_matrix, DEVICE)
96
97     siam_f1_val = evaluate_f1(y_val_enc, siam_val_pred)
98     siam_f1_test = evaluate_f1(y_test_enc, siam_test_pred)
99     print(f"B[] F1 (weighted) на val: {siam_f1_val:.4f}")
100    print(f"B[] F1 (weighted) на test: {siam_f1_test:.4f}")

```

Листинг А.6 — Центроидный классификатор. Поиск

```

1
2     search_log_siam = []
3     categories_str = ', '.join(CLASS_NAMES)
4
5     print("=" * 70)
6     print("B[] Поиск товаров по запросу индекс(:
products_siamese_e5.h5)")
7     print(f"Доступные категории: {categories_str}")
8     print("Введите стоп'' в любом поле для выхода.")
9     print("=" * 70)
10
11     siamese.eval()
12
13     while True:
14         query = input("\Запрос товара: ").strip()
15         if query.lower() in ('стоп', 'stop', 'exit', 'quit', ''):
16             break
17
18         print(f"    Доступные категории: {categories_str}")
19         print("    или( введите все'' для поиска по всем категориям)"
)

```

```

20     cat_input = input(" Категория: ").strip()
21     if cat_input.lower() in ('стоп', 'stop', 'exit', 'quit'):
22         break
23     if cat_input.lower() != 'все' and cat_input not in
CLASS_NAMES:
24         print(f" Категория '{cat_input}' не найдена. Доступны:
{categories_str}")
25         continue
26
27     # Запрос кодируем как passage: так( обучалась сеть – левая
ветвь получала passageэмбеддинги:-)
28     raw_emb = encode_passages([query])
29     with torch.no_grad():
30         t = torch.tensor(raw_emb, dtype=torch.float32).
to(DEVICE)
31         query_emb = siamese.encode(t).cpu().numpy()[0] #
вектор в пространстве Файла 3
32
33     # Поиск в Файле 3: и запрос, и товары прошли через siamese.
encode()
34     results, elapsed = search_products(
35         query_emb, cat_input,
36         prod_X_siam, # Файл 3: E5+Siameseэмбеддинги- товаров
37         prod_nm_ids, prod_categories, prod_ratings, prod_names,
prod_descs
38     )
39     if results.empty:
40         print(f" Нет товаров в категории '{cat_input}'.")
41         continue
42
43     print_search_results(results, elapsed, cat_input)
44     search_log_siam.append({'query': query, 'category':
cat_input, 'elapsed_ms': elapsed * 1000})
45
46     save_search_log(search_log_siam, 'search_log_siamese_e5.h5')

```

Листинг А.7 — Графический интерфейс на Gradio

```

1
2 #

```

```

3 # 1. Сиамская сеть
4 #
-----
5
6 class SiameseNet(nn.Module):
7     """
8     Общий энкодер + classification head.
9     emb_dim и hidden берутся из config.EMB_DIMS / config.
10    SIAMESE_HIDDEN.
11    """
12    def __init__(self, emb_dim: int, hidden: int):
13        super().__init__()
14        self.encoder = nn.Sequential(
15            nn.Linear(emb_dim, hidden),
16            nn.ReLU(),
17            nn.Linear(hidden, hidden),
18        )
19        self.head = nn.Sequential(
20            nn.Linear(hidden * 4, 256),
21            nn.ReLU(),
22            nn.Dropout(0.2),
23            nn.Linear(256, 1),
24        )
25
26    def encode(self, x: torch.Tensor) -> torch.Tensor:
27        return self.encoder(x)
28
29    def forward(self, a: torch.Tensor, b: torch.Tensor) -> torch.
30    Tensor:
31        h_a = self.encode(a)
32        h_b = self.encode(b)
33        combined = torch.cat([h_a, h_b, torch.abs(h_a - h_b), h_a *
34        h_b], dim=1)
35        return self.head(combined).squeeze(1)
36
37 #
-----
38
39 # 2. Внутренние кеши
40 #

```

```

38
39 _cache_st_model:      dict = {}      # SentenceTransformer объекты
40 _cache_prod_raw:      dict = {}      # данные из products_raw_*.h5
41 _cache_prod_siam:     dict = {}      # данные из products_siamese_*.h5
42 _cache_clf_lr:        dict = {}      # LogisticRegression
43 _cache_le:            dict = {}      # LabelEncoder
44 _cache_centroids:     dict = {}      # centroid_matrix + CLASS_NAMES
45 _cache_siamese_net:   dict = {}      # SiameseNet (eval mode)
46
47 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
48
49
50 #
-----
51 # 3. Вспомогательные функции загрузки
52 #
-----
53
54 def _decode(arr) -> np.ndarray:
55     """Декодирует массив bytes → str для( HDF5полей-, сохранённых
    как байты)."""
56     if len(arr) > 0 and isinstance(arr[0], (bytes, np.bytes_)):
57         return np.array([x.decode("utf-8") for x in arr], dtype=
    object)
58     return arr
59
60
61 def _load_st_model(model_key: str) -> SentenceTransformer:
62     if model_key not in _cache_st_model:
63         name = config.MODEL_NAMES[model_key]
64         print(f"[backend] Загрузка SentenceTransformer: {name} ...")
65     )
66     _cache_st_model[model_key] = SentenceTransformer(name)
67     print(f"[backend] {model_key} загружена.")
68     return _cache_st_model[model_key]
69
70 def _load_products_raw(model_key: str) -> dict:

```

```

71     """Загружает Файл 2 сырые( эмбединги товаров) для заданной
    модели."""
72     if model_key not in _cache_prod_raw:
73         path = config.PRODUCTS_RAW[model_key]
74         if not os.path.exists(path):
75             raise FileNotFoundError(
76                 f"Файл эмбедингов товаров не найден: {path}\n"
77                 f"Скопируйте его в папку data/ проекта."
78             )
79         print(f"[backend] Загрузка индекса товаров (raw): {path}
    ...")
80         with h5py.File(path, "r") as f:
81             X = f["X"][:]
82             nm_ids = f["nm_ids"][:]
83             categories = _decode(f["categories"][:] if "
categories" in f
84                                     else f["categories"].asstr()[:])
85             ratings = f["ratings"][:]
86             names = _decode(f["names"][:] if "names" in f
87                                     else f["names"].asstr()[:])
88             descriptions = _decode(f["descriptions"][:] if "
descriptions" in f
89                                     else f["descriptions"].asstr()
90                                     [:])
91             # Нормализуем для ускорения cosine_similarity → dot product
92             norms = np.linalg.norm(X, axis=1, keepdims=True)
93             norms[norms == 0] = 1.0
94             X_norm = (X / norms).astype(np.float32)
95
96             _cache_prod_raw[model_key] = {
97                 "X": X_norm, "nm_ids": nm_ids, "categories": categories
98                 ,
99                 "ratings": ratings, "names": names, "descriptions":
descriptions,
100             }
101             print(f"[backend] Загружено товаров (raw): {len(X)}")
102             return _cache_prod_raw[model_key]
103
104 def _load_products_siam(model_key: str) -> dict:
105     """Загружает Файл 3 эмбединги( в пространстве сиамской сети)."""

```

```

"""
106     if model_key not in _cache_prod_siam:
107         path = config.PRODUCTS_SIAM[model_key]
108         if not os.path.exists(path):
109             raise FileNotFoundError(
110                 f"Файл сиамских эмбедингов товаров не найден: {
path}\n"
111                 f"Скопируйте его в папку data/ проекта."
112             )
113         print(f"[backend] Загрузка индекса товаров (siamese): {path
} ...")
114         with h5py.File(path, "r") as f:
115             X = f["X"][:]
116             # Метаданные берём из rawфайла- (nm_ids, категории и
тд.. те же)
117             raw = _load_products_raw(model_key)
118
119             norms = np.linalg.norm(X, axis=1, keepdims=True)
120             norms[norms == 0] = 1.0
121             X_norm = (X / norms).astype(np.float32)
122
123             _cache_prod_siam[model_key] = {
124                 "X": X_norm,
125                 "nm_ids": raw["nm_ids"],
126                 "categories": raw["categories"],
127                 "ratings": raw["ratings"],
128                 "names": raw["names"],
129                 "descriptions": raw["descriptions"],
130             }
131             print(f"[backend] Загружено товаров (siamese): {len(X)}")
132         return _cache_prod_siam[model_key]
133
134
135 def _load_clf_lr(model_key: str):
136     """Загружает обученный LogisticRegression из joblibфайла-."""
137     if model_key not in _cache_clf_lr:
138         path = config.CLF_LR_PATH[model_key]
139         if not os.path.exists(path):
140             raise FileNotFoundError(
141                 f"Модель логистической регрессии не найдена: {path
}\n"
142                 f"Экспортируйте clf_lr из ноутбука командой:\n"

```

```

143         f" import joblib; joblib.dump(clf_lr, '{path}')"
144     )
145     _cache_clf_lr[model_key] = joblib.load(path)
146     return _cache_clf_lr[model_key]
147
148
149 def _load_label_encoder(model_key: str):
150     """Загружает LabelEncoder из joblib файла-."""
151     if model_key not in _cache_le:
152         path = config.LABEL_ENCODER_PATH[model_key]
153         if not os.path.exists(path):
154             raise FileNotFoundError(
155                 f"LabelEncoder не найден: {path}\n"
156                 f"Экспортируйте le из ноутбука командой:\n"
157                 f" import joblib; joblib.dump(le, '{path}')"
158             )
159         _cache_le[model_key] = joblib.load(path)
160     return _cache_le[model_key]
161
162
163 def _load_centroids(model_key: str) -> dict:
164     """
165     Строит или загружает матрицу центроидов из reviews_embeddings_
166     *.h5.
167     Центроид каждой категории = среднее эмбедингов всех отзывов
168     этой категории.
169     """
170     if model_key not in _cache_centroids:
171         # Для центроидов нужен LabelEncoder, чтобы знать
172         CLASS_NAMES
173         le = _load_label_encoder(model_key)
174         path = config.REVIEWS_EMB[model_key]
175         if not os.path.exists(path):
176             raise FileNotFoundError(
177                 f"Файл отзывных эмбедингов не найден: {path}\n"
178                 f"Он нужен для построения центроидов Метод( Б)."
179             )
180         print(f"[backend] Построение центроидов из {path} ...")
181         with h5py.File(path, "r") as f:
182             X = f["X"][:]
183             y_root = f["y_root"].asstr()[:]

```

```

182         class_names = le.classes_.tolist()
183         centroids = []
184         for cls_name in class_names:
185             mask = y_root == cls_name
186             if mask.sum() == 0:
187                 centroids.append(np.zeros(X.shape[1], dtype=np.
float32))
188             else:
189                 centroids.append(X[mask].mean(axis=0).astype(np.
float32))
190
191         centroid_matrix = np.array(centroids, dtype=np.float32)
192         # Нормализуем центроиды
193         norms = np.linalg.norm(centroid_matrix, axis=1, keepdims=
True)
194         norms[norms == 0] = 1.0
195         centroid_matrix = centroid_matrix / norms
196
197         _cache_centroids[model_key] = {
198             "matrix": centroid_matrix,
199             "class_names": class_names,
200         }
201         print(f"[backend] Центроиды построены: {len(class_names)}
категорий.")
202         return _cache_centroids[model_key]
203
204
205 def _load_siamese_net(model_key: str) -> SiameseNet:
206     """Загружает веса сиамской сети из .pt файла."""
207     if model_key not in _cache_siamese_net:
208         path = config.SIAMESE_WEIGHTS[model_key]
209         if not os.path.exists(path):
210             raise FileNotFoundError(
211                 f"Веса сиамской сети не найдены: {path}\n"
212                 f"Экспортируйте модель из ноутбука командой:\n"
213                 f" torch.save(siamese.state_dict(), '{path}')"
214             )
215         emb_dim = config.EMB_DIMS[model_key]
216         hidden = config.SIAMESE_HIDDEN[model_key]
217         net = SiameseNet(emb_dim=emb_dim, hidden=hidden)
218         net.load_state_dict(torch.load(path, map_location=DEVICE))
219         net.to(DEVICE)

```

```

220         net.eval()
221         _cache_siamese_net[model_key] = net
222         print(f"[backend] Сиамская сеть {model_key} загружена (
device={DEVICE}).")
223         return _cache_siamese_net[model_key]
224
225
226 #
-----

227 # 4. Функции кодирования текста
228 #
-----

229
230 def _encode_query(text: str, model_key: str) -> np.ndarray:
231     """Кодирует пользовательский запрос с учётом специфики модели."""
232     st = _load_st_model(model_key)
233     if model_key == "E5-large":
234         emb = st.encode(
235             ["query: " + text],
236             normalize_embeddings=True,
237             show_progress_bar=False,
238         )
239     else:
240         emb = st.encode([text], show_progress_bar=False)
241     return emb[0].astype(np.float32)
242
243
244 def _project_siamese(emb: np.ndarray, model_key: str) -> np.ndarray
:
245     """Прогоняет эмбединг через encoder сиамской сети."""
246     net = _load_siamese_net(model_key)
247     with torch.no_grad():
248         t = torch.tensor(emb, dtype=torch.float32).unsqueeze(0).
to(DEVICE)
249         out = net.encode(t).cpu().numpy()[0]
250         # Нормализуем
251         norm = np.linalg.norm(out)
252         return (out / norm).astype(np.float32) if norm > 0 else out
253

```

```

254
255 #
-----

256 # 5. Поиск товаров
257 #
-----

258
259 def _cosine_top_k(query_vec: np.ndarray, index: dict,
260                  category_filter: str, top_k: int = 10):
261     """
262     Возвращает top_k товаров по косинусному сходству.
263     query_vec должен быть нормализован.
264     """
265     X          = index["X"]
266     nm_ids     = index["nm_ids"]
267     categories = index["categories"]
268     ratings   = index["ratings"]
269     names     = index["names"]
270     descs     = index["descriptions"]
271
272     if category_filter.strip().lower() == "все":
273         idx_pool = np.arange(len(X))
274     else:
275         idx_pool = np.where(categories == category_filter)[0]
276
277     if len(idx_pool) == 0:
278         return []
279
280     q_norm = query_vec / (np.linalg.norm(query_vec) + 1e-9)
281     sims   = X[idx_pool] @ q_norm          # dot product = cosine
нормализовано()
282     top    = np.argsort(sims)[::-1][:top_k]
283     real   = idx_pool[top]
284
285     results = []
286     for i, ri in enumerate(real):
287         results.append({
288             "rank":      i + 1,
289             "nm_id":     int(nm_ids[ri]),
290             "name":      str(names[ri]),

```

```
291         "rating":      float(ratings[ri]),
292         "similarity": float(sims[top[i]]),
293         "description": str(descs[ri][:200],
294     })
295     return results
```